



Anomaly Detection in Smart Meters

by

Zak Osman

This thesis has been submitted in partial fulfillment for the
degree of Bachelor of Science in Software Development in Computer Systems

in the
Faculty of Engineering and Science
Department of Computer Science

May 2025

Declaration of Authorship

This thesis titled, "Anomaly Detection in Smart Meters" and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for an undergraduate degree at Munster Technological University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this university or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis report is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: Zak Osman

Date: 14/09/2024

MUNSTER TECHNOLOGICAL UNIVERSITY CORK

Abstract

Faculty of Engineering and Science

Department of Computer Science

Bachelor of Science in Software Development

by Zak Osman

This project aims to develop an AI-powered anomaly detection system for Cork City Council to manage energy consumption data from smart meters installed in their buildings. The system will process and analyse hourly data collected over three years from 14 smart meters measuring oil tank levels in Council buildings. Within this application, the Council will be able to monitor energy consumption patterns, identify anomalies in usage, and receive alerts for potential issues such as unexpected consumption during off-hours or equipment malfunctions.

The system will employ machine learning techniques to adapt to each building's unique consumption patterns, considering factors such as time of day, day of the week, and seasonal variations. This approach will help minimise false positives while ensuring accurate detection of true anomalies. The application will feature a user-friendly interface for visualising consumption data and detected anomalies, making it accessible for non-technical Council staff.

Acknowledgements

The acknowledgements and the people to thank go here, don't forget to include your project supervisor (term one and two)...

Contents

Declaration of Authorship	i
Abstract	ii
Acknowledgements	iii
List of Figures	viii
List of Tables	x
Abbreviations	xi
1 Introduction	1
1.1 Motivation	1
1.1.1 The Significance of Energy Smart Meters: A Case Study	3
1.1.2 My Contribution	5
1.2 Structure of This Document	7
2 Background	8
2.0.1 Thematic Area within Computer Science	8
2.1 A Review of Anomaly Detection in Smart Meters	11
2.1.0.1 1. Lee et al. (2022): Anomaly Detection for Power Man- agement with Battery Storage Systems	12
2.1.0.1 2. Chahla et al. (2019): Hybrid Model for Anomaly Detec- tion in Power Consumption Data	13
2.1.0.2 3. Aligholian et al. (2019): Unsupervised Learning for Online Abnormality Detection in Smart Meter Data	13
2.1.0.3 4. Vafeiadis et al. (2019): Anomaly Detection in Smart Meters Using Autoencoders	14
2.1.0.4 5. Khoshsefat et al. (2024): CNN-Based Approach for District-Level Anomaly Detection in Tehran	14
2.1.0.5 6. Singhal et al. (2022): Anomaly Detection in Smart Meters - An Analytical Study	15

2.1.0.6	7. Artificial Intelligence-Based Anomaly Detection of Energy Consumption in Buildings: A Review, Current Trends, and New Perspectives	15
2.1.1	Python Libraries for Anomaly Detection	16
2.1.1.1	7. Comparison and Insights	18
2.1.1.2	Future Directions and Trends	20
3	Problem - Anomaly Detection in Smart Metres	21
3.1	Problem Definition	21
3.1.1	Current Scenario	21
3.1.2	Significance of Anomaly Detection	21
3.1.3	Technical and Financial Impacts	22
3.1.4	Proposed Focus	22
3.2	Objectives	22
3.3	Functional Requirements	23
3.4	Non-Functional Requirements	23
3.5	Plan for Next Semester	25
4	Implementation Approach	27
4.1	Methodology	27
4.1.0.1	Phase 1: Research and Requirement Gathering	27
4.1.0.2	Phase 2: System Design and Planning	27
4.1.0.3	Phase 3: Prototyping and Testing Plan	27
4.1.0.4	Phase 4: Evaluation and Iteration Planning	28
4.1.0.5	Research Method	28
4.1.0.6	Developing Necessary Technical Skills	29
4.1.0.7	Project Management Approach	30
4.2	Architecture	31
4.2.1	Technologies Involved	32
4.2.2	Database Overview	34
4.2.3	Tools and Technologies	34
4.3	Risk Assessment	35
4.4	Implementation Plan Schedule	37
4.4.1	Semester 1: Research and Initial Prototyping	37
4.4.2	Semester 2: Full Development and Testing	37
4.5	Prototype	38
4.5.1	Envisioned Interface and Data Flow	38
4.5.2	Anomaly Detection Concept	38
4.5.3	Deployment Plans	38
4.5.4	Next Steps in Prototyping	39
4.6	Implementation Progress	39
4.7	Evaluation	42
4.7.1	Quantitative Metrics	42
4.7.2	Qualitative Evaluation	42
4.7.3	Comparison with Initial Goals	43
4.7.4	4.8 Planned Implementation Details	44
4.7.4.1	4.8.1 Initial Testing Strategy	44

4.7.4.2	4.8.2 Machine Learning Models for Advanced Detection . . .	44
4.7.4.3	4.8.3 Statistical Methods for Complementary Analysis . . .	45
4.7.4.4	4.8.4 Implementation Framework	45
4.7.4.5	4.8.5 Scalability and Future Integration	45
4.8	Conclusions and Future Work	45
4.9	Discussion	45
4.10	Future Work	49
5	Implementation	50
5.1	Difficulties Encountered	52
5.1.1	Internal Difficulties	52
5.1.2	Data Quality Issues	52
5.1.3	Anomaly Detection Logic	53
5.1.4	False Positive Reduction	54
5.1.5	Refill Pattern Detection	55
5.1.6	Machine Learning Model Performance	56
5.1.7	Weather Data Integration	57
5.2	Real-time Visualisation Performance	57
5.2.1	Context-Aware Detection	58
5.3	External Difficulties	59
5.3.1	Course Workload	59
5.3.2	Ramadan	59
5.3.3	Sickness	59
5.4	Actual Solution Approach	60
6	Testing and Evaluation	67
6.1	System Testing	68
6.1.1	Testing Environment	68
6.1.2	System Flow Overview	69
6.1.3	Experimental Design	71
6.1.4	Testing Objectives	71
6.1.5	Visual Results	72
6.1.6	Incorporating Weather Context	75
6.1.7	Anomaly Severity Levels Visualization	76
6.1.8	Refill Days vs. Anomaly Detection	77
6.1.9	Saved Models	77
6.2	Results	79
7	Discussion and Conclusions	80
7.1	Project Overview and Review	80
7.2	Skills Gained	81
7.2.1	Python, Dash, Plotly	81
7.2.2	Machine Learning Models	81
7.2.3	Project Management	82
7.2.4	Area of Improvement	83
7.2.5	User Feedback	84
7.2.5.1	Positive Feedback	84

7.3 Conclusion	85
7.4 Future Work	86
Bibliography	87
A Code Snippets	89
B Wireframe Models	93

List of Figures

3.1	Gantt Chart: Project Plan for Next Semester	25
4.1	Scrumban Methodology [1]	30
4.2	High Level Architecture Diagram for Web Application [2]	31
4.3	Complete System Architecture for Anomaly Detection System	34
4.4	Interactive Dashboard Output	40
5.1	Example of Incorrect Anomaly Flags in Early Implementation	55
5.2	Example of the constant call back errors	57
5.3	Trello board showing task progression across sprints using a Scrumban approach.	62
6.1	Flowchart showing deployment and integration of the anomaly detection system.	69
6.2	Overall architecture of the Smart Meter Anomaly Detection System showing component interactions and data sources	70
6.3	Console output showing data aggregation, feature extraction, and threshold calculation during system preparation for testing.	72
6.4	Example of a detected anomaly using domain knowledge rules.	72
6.5	Dashboard view of real anomalies detected)	73
6.6	Interactive visual with weekend highlighting	73
6.7	Average consumption by day of week and by month	74

6.8	Detailed anomaly explanation.	74
6.9	Daily gas consumption and corresponding temperature for Castletown- bere Area Engineers Office.	75
6.10	severity level.	76
6.11	severity level.	77
6.12	previously saved models	77
6.13	Logged anomalies in the <code>smaart_meter_alarm</code> MySQL table showing anomaly scores, timestamps, severity and meter IDs.	78
A.1	Function to establish and return a MySQL database connection using the stored configuration	89
A.2	Function to log anomaly data to the database after validating the anomaly date and parsing fields	90
A.3	LSTM autoencoder model with Batch Normalization and Dropout for enhanced anomaly detection	90
A.4	Global dictionary mapping meter IDs to corresponding building names for identification	91
A.5	Daily anomaly detection logic using z-score and day-of-week pattern with building-specific thresholds	91
A.6	Function to apply domain knowledge rules by classifying building types and analysing consumption patterns	92
A.7	Function to generate a human-readable explanation of anomalies based on consumption deviation	92
B.1	Wireframe layout of the control panel interface	93
B.2	Wireframe showing data panel with anomaly and refill indicators	93
B.3	Wireframe showing anomaly statistics and consumption summary panel	94
B.4	Wireframe showing average weekly and monthly consumption details	94
B.5	Wireframe showing anomaly details including consumption metrics and detected issues	94

List of Tables

2.1	Anomaly Detection Model Parameters	11
4.1	Updated Risk Matrix	37
5.1	Summary of Internal Difficulties and Their Effects on System Design . . .	52
5.2	Summary of External Difficulties and Their Effects on Project Progress .	59
6.1	Summary of Anomaly Detection Methods Used	79

Abbreviations

LAH List Abbreviations **Here**

For/Dedicated to/To my...

Chapter 1

Introduction

1.1 Motivation

As a computer science student in Ireland, I've always been fascinated by the power of Artificial Intelligence and its potential to solve complex real-world problems. When it came time to choose my final year project, I knew I wanted to explore Machine Learning's capabilities in a meaningful way. My search for a challenging and impactful application led me to the field of energy management, specifically the use of smart meters in Cork County.

The European Union's Green Deal sets ambitious targets for energy efficiency, aiming for a 50% improvement by 2030 in the public sector. Smart meters play a pivotal role in achieving these targets by providing real-time data that can optimise energy usage, reduce waste, and promote sustainability. Cork County Council, for instance, has deployed 14 smart meters across its buildings, generating over 300,000 hourly readings in three years. This vast dataset represents an opportunity to contribute to the Green Deal's goals through Machine Learning-powered anomaly detection.

The significance of energy efficiency is underscored by Ireland's commitments to reduce carbon emissions, meet EU directives, and improve energy management within the public sector. Detecting anomalies in energy consumption data not only ensures accurate billing but also identifies inefficiencies and potential malfunctions that can result in unnecessary costs and emissions. Addressing these challenges through advanced Machine learning techniques aligns with national sustainability goals while demonstrating the practical value of Machine Learning in real-world scenarios.

When I was introduced to this project by my supervisor, I learned that Cork County Council had installed 14 smart meters to measure oil tank levels in their buildings,

generating vast amounts of data over the past three years. This immediately caught my attention, as I saw an opportunity to apply Machine Learning techniques to make a real difference in local energy management. With over 300,000 hourly readings collected, I realized that Machine Learning could play a crucial role in analyzing this data effectively.

The more I delved into the subject, the more excited I became about the possibilities. I found that detecting anomalies in smart meter data isn't just a fascinating technical challenge – it's a critical component in Cork's efforts to enhance energy efficiency, reduce costs, and promote sustainable practices. The idea of using Machine Learning to identify irregular patterns that could indicate unexpected consumption or equipment faults really appealed to my problem-solving nature.

What truly motivated me was the potential impact of this project on Cork County Council's operations. I learned that unexpected energy consumption could lead to higher costs and inefficiencies. By developing Machine Learning powered anomaly detection techniques, I saw an opportunity to not only address this issue but also contribute to maintaining the integrity and reliability of the council's energy systems.

As I researched further, I became increasingly aware of Ireland's move towards greener energy solutions to meet EU directives. This transition, I realized, could greatly benefit from advanced Machine Learning techniques for analyzing unusual consumption patterns, even in oil-based heating systems. The prospect of empowering Cork County Council with Machine Learning-derived insights into their energy usage habits further fueled my enthusiasm for the project.

Driven by my passion for Machine Learning and its applications, I decided to focus my final year project on developing robust anomaly detection techniques using artificial intelligence and optimization algorithms. My goal is to leverage the extensive data collected by Cork's smart meters and enhance the accuracy and efficiency of anomaly detection, particularly for identifying consumption during unexpected times like overnight or weekends.

I'm thrilled by the challenge of applying cutting-edge Machine Learning technologies to foster a more sustainable and economically efficient energy landscape in Cork. This project not only allows me to dive deep into Machine Learning applications but also aligns with our local and national goals for energy sustainability. As I embark on this journey, I'm excited by the possibility that my work with Machine Learning could contribute to positioning Cork and Ireland as leaders in innovative energy management within the EU.

This project represents the perfect synthesis of my interests in Machine Learning and my desire to make a tangible impact in my local community. It's not just an academic

exercise for me – it’s an opportunity to showcase the transformative power of Machine Learning in addressing one of the most pressing challenges of our time, right here in Cork. By developing a system that can accurately detect anomalies while minimizing false positives, I hope to provide Cork County Council with a valuable tool for optimizing their energy consumption and reducing waste.

1.1.1 The Significance of Energy Smart Meters: A Case Study

The significance of energy efficiency in public sector buildings cannot be overstated. Energy smart meters, such as those deployed in Cork County, provide invaluable insights into energy consumption patterns, helping to identify inefficiencies and reduce unnecessary energy usage. While this case study focuses on Cork County, the methodology and findings are extensible and applicable to other local authorities, making the research relevant on both national and international levels.

Smart meters play a critical role in achieving the goals outlined in the 2030 Green Deal, which aims to reduce carbon emissions and improve energy efficiency across the European Union. Public sector buildings are a key area of focus, as they account for a significant proportion of energy consumption. The EU’s Green Deal has set a target of achieving a 50% improvement in public sector energy efficiency by 2030, which is not only ambitious but also a necessary step toward reducing carbon emissions and creating a more sustainable future for generations to come.

The dataset analyzed in this project includes over 300,000 hourly readings and approximately 75,000 monthly bills collected from 14 smart meters installed in public sector buildings. These readings provide a comprehensive view of energy usage patterns, enabling the identification of irregularities and inefficiencies. Accurate data analysis ensures that actionable insights can be derived, making it possible to address energy waste effectively.

Smart meters are central to this transition. These devices provide real-time data, enabling organizations to monitor energy consumption closely, identify inefficiencies, and take proactive measures to reduce waste. They also ensure accurate billing and can help detect unusual patterns, such as unexpected energy use during weekends or overnight. For Cork County Council, addressing anomalies in energy usage is about more than cutting costs—it’s about contributing to Ireland’s broader sustainability goals and ensuring public resources are utilized responsibly.

By analyzing energy usage through advanced detection algorithms, this research directly supports these objectives by identifying and mitigating energy waste. The findings from

this study are not limited to Cork County but provide a scalable model that can be adapted for use by other local authorities aiming to optimize energy efficiency and sustainability. This extensibility demonstrates the relevance of the project on a broader scale, highlighting its potential impact beyond the immediate context.

This project aims to make a tangible contribution to these efforts by developing an Machine Learning-powered anomaly detection system tailored to the specific needs of Cork County Council. By identifying irregularities in energy consumption and minimizing false positives, the system can empower decision-makers to act promptly and efficiently. In doing so, this research aligns with Ireland's commitments to the EU Green Deal, offering a practical solution to enhance energy efficiency, reduce operational costs, and support the nation's transition toward a carbon-neutral future.

1.1.2 My Contribution

This project contributes to the fields of energy management and computer science by developing a tailored anomaly detection system for Cork County Council's oil consumption data. It addresses the practical challenges of managing large datasets and improving energy efficiency in public sector buildings. The key contributions are:

1. Data Processing and Analysis:

- Handling and analyzing over 300,000 hourly readings from 14 smart meters.
- Implementing robust data cleaning and normalization techniques to ensure accurate and meaningful time-series analysis.

2. Anomaly Detection System:

- Developing custom algorithms to identify unexpected consumption patterns, particularly during non-standard hours like nights and weekends.
- Minimizing false positives to ensure reliability and maintain stakeholder trust.

3. Machine Learning Application:

- Applying supervised and unsupervised learning techniques to uncover patterns in energy consumption data.
- Incorporating time-series analysis to account for seasonal variations and daily trends in oil usage.

4. Real-Time Alert System:

- Designing a real-time notification system to promptly alert stakeholders of detected anomalies.
- Prioritizing alerts based on severity and confidence levels, enabling swift and effective responses.

5. User Interface Development:

- Creating an intuitive dashboard to visualize energy consumption patterns and detected anomalies.

- Incorporating interactive tools to simplify the interpretation of complex trends and data insights.

6. System Integration:

- Seamlessly integrating the anomaly detection system with Cork County Council's smart meter infrastructure.
- Developing APIs to enable efficient data exchange with other council systems.

7. Performance Optimization:

- Implementing efficient algorithms and data structures to process large datasets in real-time.
- Optimizing system performance to ensure continuous and timely anomaly detection.

8. Environmental Impact:

- Supporting Cork County Council's energy efficiency goals by enabling early detection of unusual consumption patterns.
- Reducing energy waste and associated costs, contributing to Ireland's sustainability targets under the European Green Deal.

This project is more than just an academic exercise. It demonstrates the practical application of machine learning and data analytics to a real-world problem, aligning with Ireland's national goals for energy efficiency and sustainability. By focusing on Cork County Council's specific needs, this work creates a scalable model that other public-sector organizations can adopt to optimize energy use and reduce their environmental impact.

1.2 Structure of This Document

The structure of this document is as follows:

- Chapter 1 introduces the project, explaining why we're developing a system to spot unusual oil use in Cork County Council buildings, what the project adds to the field, and what it aims to do.
- Chapter 2 contains research on the main topics of the project, looking at what's been done before in energy management, smart meters, and ways to find unusual patterns in data.
- Chapter 3 outlines the problem this project is trying to solve, what we want to achieve, what the system needs to do, and other important features it should have.
- Chapter 4 describes how we're going to build the system, including its overall design, how it will be used, what risks we might face, and our plan for making it. It also shows what the system might look like when it's done.
- Chapter 5 explains how we actually built the system, including how we handled the data, what methods we used to spot unusual oil use, and how we made the alert system.
- Chapter 6 shows how well the system works, including how accurate it is at finding unusual oil use and how good it is at not raising false alarms.
- Chapter 7 wraps up the document by talking about what we learned, what we achieved, and what could be done next to make the system even better.

Each chapter builds on the one before, giving a full picture of the project from start to finish.

Chapter 2

Background

2.0.1 Thematic Area within Computer Science

This section provides an in-depth exploration of the thematic areas relevant to the project, with a specific focus on the intersection of data analysis, machine learning, energy management systems, and time-series analysis. These domains collectively underpin the development of an Machine Learning-powered anomaly detection system tailored for energy consumption data from smart meters.

When I began this project, I had no prior knowledge of this field. My plan was to start broadly by exploring the wider domain of anomaly detection, gradually narrowing my focus to smart meters and energy efficiency as I gained a deeper understanding. This approach not only allowed me to build a foundational grasp of the concepts but also helped me identify key challenges and opportunities that have shaped my research direction. By understanding the theoretical and practical foundations in these areas, this section aims to set the stage for the methodologies and technologies employed in subsequent chapters. The following subsections delve into each thematic area, highlighting their significance and relevance to this research project.

Data Analysis: Data analysis is the process of inspecting, cleansing, transforming, and modeling data to discover useful information, inform conclusions, and support decision-making. In the context of this project, it involves processing and interpreting large volumes of oil consumption data from smart metres.

Machine Learning: Machine learning is a branch of artificial intelligence focused on building applications that learn from data and improve their accuracy over time without being explicitly programmed. For this project, machine learning techniques are crucial in developing algorithms to detect anomalies in oil consumption patterns.

Energy Management Systems: Energy Management Systems (EMS) are tools used to monitor, control, and optimize the performance of energy generation and/or consumption. In this project, we're focusing on the consumption aspect, specifically oil usage in Cork County Council buildings.

Time Series Analysis: Time series analysis comprises methods for analyzing time series data to extract meaningful statistics and characteristics. This is particularly relevant for our project as we're dealing with oil consumption data collected over time from smart metres.

Community:

The main areas that should be watched with this project are Data Analysis, Machine Learning in Energy Management, and Time Series Anomaly Detection.

Standards: One of the main things to be watched is ISO standards related to energy management and data analysis. For instance, ISO 50001 is about energy management systems, which is relevant to our project's goals of improving energy efficiency.

Books:

1. "Data Mining: Concepts and Techniques" by Jiawei Han, Micheline Kamber, and Jian Pei
2. "Machine Learning for Time-Series with Python" by Ben Auffarth
3. "Energy Management Systems & Direct Digital Control" by Richard P. Fennelly

Conferences:

1. International Conference on Machine Learning (ICML)
2. KDD Conference on Knowledge Discovery and Data Mining
3. International Conference on Energy Efficiency in Buildings

Possible Interested Organizations:

1. Cork County Council
2. Other local governments and municipalities
3. Energy management companies

4. Environmental agencies
5. Smart city technology providers

This project could be useful for organizations looking to optimize their energy consumption, reduce costs, and improve their environmental footprint. The increasing focus on energy efficiency and the growing adoption of smart metering technologies make this type of Anomaly detection system increasingly relevant.

In recent years, there has been a significant shift towards digitization and automation in energy management. Organizations are transitioning from traditional manual meter readings to smart metering systems that provide real-time data. This shift, while beneficial in many ways, introduces new challenges and risks that need careful management.

The installation of smart meters, like the 14 utilized by Cork County Council to measure oil tank levels, generates vast amounts of data. By 2024, these meters will have been operational for three years, collecting over 300,000 hourly readings. This wealth of data provides unprecedented insight into energy consumption patterns but also demands sophisticated analysis techniques to extract meaningful information and effectively detect anomalies.

The necessity for advanced anomaly detection in energy consumption data has become increasingly critical as anomalies can indicate a range of issues, from equipment malfunctions to energy waste and potential security breaches. For instance, unexpected energy usage during nights or weekends, as noted in the project abstract, could point to inefficiencies in building management systems or unauthorized usage, underscoring the importance of timely and accurate detection.

Moreover, with the growing imperative to combat climate change and reduce carbon footprints, efficient energy management is becoming a top priority. The European Union has established stringent targets for carbon emission reductions, with local authorities like Cork County Council playing a pivotal role in meeting these goals. Effective tools for optimizing energy usage are essential in this effort.

Anomalies in energy consumption not only lead to unnecessary costs and increased carbon emissions but can also result in reputational damage. In severe cases, undetected anomalies might pose safety risks, emphasizing the need for robust anomaly detection systems.

This project's development of an Machine Learning-powered anomaly detection system is both timely and relevant, potentially benefiting not only large organizations like Cork

County Council but also smaller entities that are increasingly incorporating smart metering technologies. As organizations digitize their energy management processes, the demand for sophisticated, Machine Learning-driven analysis tools is expected to grow.

The insights from this project could also enhance smart city technologies. Urban areas are increasingly dependent on data-driven decision-making, and tools capable of analyzing and interpreting vast amounts of sensor data are crucial. The anomaly detection techniques developed here, particularly those using Python libraries such as Pandas for data manipulation and scikit-learn for implementing machine learning algorithms, could be adapted for broader applications in urban infrastructure monitoring.

In conclusion, this project addresses a critical need in energy management and aligns with broader trends towards sustainability, digitization, and data-driven decision-making. Its potential applications extend beyond the immediate context of Cork County Council, making it a relevant and timely contribution to the field of computer science and its applications in energy management.

2.1 A Review of Anomaly Detection in Smart Meters

Anomaly detection within energy consumption is critical for identifying irregular patterns that could indicate issues such as energy wastage, equipment malfunction, or unusual user behavior. This section examines various techniques and methodologies specifically tailored for anomaly detection in energy usage data, providing insights into their application within smart meter datasets.

Parameter	Value
K-means clusters	11
LSTM hidden layers	3
LSTM learning rate	0.0001
LSTM dropout rate	0.2
LSTM batch size	64
LSTM epochs	40
Anomaly threshold	13
Auto-Encoder layers	[24, 50, 20, 2, 20, 50, 24]
Auto-Encoder threshold	0.04
Data resolution	1 hour

TABLE 2.1: Anomaly Detection Model Parameters

A Literature Review of Anomaly Detection in Energy Consumption Data

The field of anomaly detection in smart metering systems has seen a significant rise in research interest, primarily due to the increasing complexity of energy management

and the need to optimize power consumption. Various approaches have been proposed to enhance the detection of anomalies, ranging from deep learning methods to classical machine learning models. In this literature review, we examine six papers that have made notable contributions in this area.

2.1.0.1 1. Lee et al. (2022): Anomaly Detection for Power Management with Battery Storage Systems

Lee et al. introduced an innovative smart metering technique capable of real-time anomaly detection, specifically designed for home power management. Their approach employs a combination of Graph Convolutional Networks (GCN) and Bidirectional Long Short-Term Memory Networks (BiLSTM) to analyze smart meter data from 900 households. By leveraging the spatial-temporal relationships within the data, their model detects outliers and missing values, which are common in smart meter data due to equipment faults or communication issues.

The GCN-BiLSTM network's design is tailored to capture both spatial correlations between different meters and temporal correlations within the same meter's time series data. This dual approach ensures a more accurate detection of anomalies compared to traditional methods that only consider one dimension. The reconstruction error of the model is calculated as:

$$e = ||x_t - \hat{x}_t||^2 \quad (2.1)$$

where x_t is the actual input data at time t , and $\hat{x}_t$ is the reconstructed data point. Anomalies are detected when the error e exceeds a predefined threshold θ . The integration of this anomaly detection model into a power management system results in significant reductions in electricity costs and grid dependency.

Unlike earlier studies, Lee et al. incorporated a battery storage system (BSS) and electric vehicle (EV) charging strategies to further optimize power consumption and manage peak load scenarios. This optimization is achieved through multi-objective functions that simultaneously maximize the profits of renewable energy providers and minimize peak power consumption:

$$a = \arg \min_{i=1}^k \sum_{x \in C_i} ||x - \mu_i||^2 \quad (2.2)$$

This approach enables more effective energy distribution, better load balancing, and enhanced integration of renewable energy sources, providing a comprehensive solution to the growing challenges in energy management.

2. Chahla et al. (2019): Hybrid Model for Anomaly Detection in Power Consumption Data

Chahla et al. \cite{Chahla2019} focused on the detection of anomalies in power consumption data using a hybrid model that combines Auto-Encoders and LSTM networks with K-means clustering. Their approach aims to identify unusual patterns in energy consumption that deviate from typical daily usage. The methodology is divided into two stages: identifying anomalous days using Auto-Encoders, and then localizing the detected anomalies within the day using LSTM and K-means clustering.

The clustering objective function used in this method is as follows:

$$\min_{PW_{grid}(t), O_{grid}(t), PW_{PV}(t)} PW_{grid}(t) \cdot O_{grid}(t) - PW_{PV}(t) \cdot O_{PV}(t) \quad (2.3)$$

where k represents the number of clusters, μ_i is the centroid of cluster C_i , and x denotes the data points. This objective function minimizes the intra-cluster variance, making it highly effective for categorizing daily consumption patterns. The localized anomalies are then flagged when the LSTM predictions deviate significantly from the actual consumption values.

Chahla et al. evaluated their model using real-world power consumption data from the Pecanstreet dataset, which includes hourly power consumption records from multiple households. Their results demonstrate that the proposed model can successfully identify both macro and micro anomalies, which are difficult to detect using standard statistical methods.

2.1.0.2 3. Aligholian et al. (2019): Unsupervised Learning for Online Anomaly Detection in Smart Meter Data

Aligholian et al. present a comprehensive study on unsupervised machine learning methods for anomaly detection in smart meter data. The paper investigates four different unsupervised techniques: prediction-based regression, prediction-based neural network, cluster-based, and projection-based methods. The authors utilize real-world data from the Pecan Street project in Austin, Texas, to evaluate the effectiveness of these methods in detecting various types of anomalies, such as abnormal power consumption trends and unusual electricity usage patterns.

The study identifies that different types of features, such as load-based, contextual, and environmental, play significant roles in influencing the detection capability of each method. Notably, historical load features are found to be more effective in prediction-based methods for identifying abnormal load trends, while cluster-based methods are more suited for pinpointing specific abnormal instances. A key contribution of this work is the implementation of online detection techniques, such as Isolation Forest (IF) and Lightweight Online Detector of Anomalies (LODA), which provide real-time anomaly detection capabilities. The research shows that IF outperforms the other methods with a Matthews Correlation Coefficient (MCC) score of 0.81. This paper emphasizes the importance of feature selection and customization for each method, which enhances the detection accuracy and efficiency.

2.1.0.3 4. Vafeiadis et al. (2019): Anomaly Detection in Smart Meters Using Autoencoders

Vafeiadis et al. conducted a large-scale study on anomaly detection in smart meters using data from 141,433 meters, producing around 40 million recorded events over one year. The focus of this research is on using autoencoders to detect multiple types of errors, such as opposite errors (e.g., power down versus power restored) and successive errors (e.g., power disconnection followed by reconnection). By utilizing machine learning techniques such as autoencoders, the authors aim to detect anomalies at both the individual meter level and across the distribution network.

The proposed model efficiently identifies patterns that deviate from expected operational behaviors, which are flagged as potential anomalies. This detection is based on statistical modeling and predefined error thresholds. The methodology supports the automation of maintenance and quick intervention in scenarios such as equipment malfunctions and electricity theft. The results show that the autoencoder model is highly effective in identifying multiple types of errors, offering a valuable tool for enhancing the reliability and security of smart meter infrastructures.

2.1.0.4 5. Khoshsefat et al. (2024): CNN-Based Approach for District-Level Anomaly Detection in Tehran

Khoshsefat et al. propose a CNN-based anomaly detection framework to distinguish between authorized and unauthorized electricity consumption in five districts of Tehran, Iran. The study utilizes data from 13,000 consumers over a one-year period and employs a binary classification model to detect anomalies. The CNN model uses features such as

average load consumption, peak load consumption, off-peak load, and monthly average consumption to classify users as authorized or unauthorized.

The model demonstrates an average accuracy of 71.3% across the districts, indicating its potential effectiveness in distinguishing between legitimate and illegitimate consumption patterns. The authors suggest that future research could enhance the generalizability of the model by using transfer learning to apply it to other regions with similar consumption patterns. This study highlights the importance of feature engineering and advanced machine learning techniques in improving anomaly detection performance in smart grids.

2.1.0.5 6. Singhal et al. (2022): Anomaly Detection in Smart Meters - An Analytical Study

Singhal et al. conducted an extensive analytical study on anomaly detection in smart meters, highlighting the importance of machine learning (ML) techniques in the effective management of electricity use. The paper provides an overview of several smart meter data analytics algorithms and their applications, focusing on detecting anomalies such as energy theft, leakage, and incorrect billing.

The study discusses different data analytics tools like R, MATLAB, and cloud-based platforms like Hadoop and Spark, and evaluates their performance in handling large datasets. Singhal et al. emphasize the role of data visualization and outlier detection techniques in uncovering anomalies that are not apparent through traditional methods. They propose a systematic approach that combines data collection, data cleaning, classification, and visualization to identify patterns indicative of abnormal energy consumption.

The authors utilize Long Short-Term Memory (LSTM) networks and Recurrent Neural Networks (RNN) for predictive modeling, demonstrating their effectiveness in sequential classification. The research findings indicate that the proposed ML-based methodology can detect consumption anomalies with high accuracy, enabling utilities to provide personalized feedback to customers and optimize grid management.

2.1.0.6 7. Artificial Intelligence-Based Anomaly Detection of Energy Consumption in Buildings: A Review, Current Trends, and New Perspectives

Himeur et al. (2021) conducted a comprehensive review of artificial intelligence techniques for detecting anomalies in building energy consumption. The study emphasizes

the role of anomaly detection in optimizing energy usage, reducing waste, and promoting sustainability.

The paper provides a detailed taxonomy of anomaly detection methods, categorizing them based on techniques such as clustering, time-series analysis, and supervised learning. It highlights key applications, including identifying excessive energy consumption, detecting appliance faults, and mitigating non-technical losses like theft or meter malfunctions. Additionally, the review discusses challenges such as the lack of annotated datasets, standardized evaluation metrics, and privacy concerns, which hinder the scalability and real-time deployment of these systems.

Himeur et al. also propose future directions, including the integration of anomaly detection with non-intrusive load monitoring and the use of contextual data like occupancy patterns to enhance system accuracy. The study serves as a foundational reference for understanding state-of-the-art methods and their limitations.

This paper is particularly relevant to the current project as it offers insights into advanced Machine Learning techniques, highlights challenges related to data handling and scalability, and aligns with the project's focus on anomaly detection for improving energy efficiency in public sector buildings. Furthermore, according to Google Scholar, this paper has been cited 520 times, indicating its significant influence and relevance within the academic and professional communities.[\[3\]](#).

2.1.1 Python Libraries for Anomaly Detection

The implementation of the anomaly detection system will leverage several Python libraries known for their robustness and wide adoption in the data science community. These libraries facilitate various functions from data manipulation to advanced machine learning models. Here I outline the primary libraries and their roles:

- **Pandas** - A powerful data analysis and manipulation library in Python. It is used for handling and preprocessing the raw data collected from the smart meters.
- **NumPy** - A library for numerical computations in Python. It is utilized for handling numerical data and performing mathematical operations efficiently during anomaly detection.
- **Plotly** - A graphing library used for creating interactive visualizations. In this project, it is integrated into Dash to display gas consumption trends and detected anomalies on the dashboard.

- **Scikit-learn** - A machine learning library in Python. It provides tools for implementing and evaluating anomaly detection models, such as Isolation Forests and clustering techniques.
- **Dash** - A library for building web-based data visualization applications. Dash is used in this project to create the interactive dashboard for visualizing processed data.
- **Flask** - A lightweight framework used to build the API for connecting the backend (Python scripts and database) with the dashboard.
- **Matplotlib** - A Python plotting library used for basic visualizations during the development phase to validate preprocessing and anomaly detection methods.
- **SQLAlchemy** - A Python SQL toolkit and ORM used for interacting with the MySQL database in the project. While MySQL is planned for scalability and integration in the later stages, the project currently uses a CSV file for data storage and processing. This approach simplifies development and allows for quick testing of the pipeline.

These tools will be integrated within my development environment to process data, evaluate models, and visualize anomalies, ensuring a comprehensive approach to tackling the challenges of anomaly detection in smart meter data.

2.1.1.1 7. Comparison and Insights

All six reviewed papers offer unique contributions to the field of anomaly detection in energy consumption data:

1. **Lee et al. (2022)** focused on integrating anomaly detection with power management strategies using GCN and BiLSTM networks.
2. **Chahla et al. (2019)** presented a hybrid model combining Auto-Encoders and LSTM networks with K-means clustering.
3. **Aligholian et al. (2019)** explored various unsupervised learning methods, emphasizing feature selection and real-time detection.
4. **Vafeiadis et al. (2019)** utilized autoencoders for large-scale network-level anomaly detection.
5. **Khoshsefat et al. (2024)** applied a CNN-based model for district-level anomaly detection.
6. **Singhal et al. (2022)** provided a comprehensive analytical study of ML-based techniques, evaluating various tools and platforms.

Conclusion:

Overall, these studies underscore the potential of combining multiple machine learning techniques to address the complexities of anomaly detection in energy systems. Future research could focus on integrating reinforcement learning and explainable AI, along with exploring transfer learning to extend the applicability of these models to diverse geographical regions and consumption patterns.

Lee et al. (2022) proposed a real-time anomaly detection framework integrating Graph Convolutional Networks (GCN) and Bidirectional Long Short-Term Memory (BiLSTM) networks, emphasizing the reduction of electricity costs and improved efficiency through the incorporation of battery storage systems and electric vehicle charging strategies [4]. Chahla et al. (2019) introduced a hybrid model combining Auto-Encoders, LSTM networks, and K-means clustering, providing a robust two-stage framework to detect and localize anomalies at both macro and micro levels. This model excels in pinpointing anomalies at specific times and locations, making it ideal for detailed consumption analysis [5].

Aligholian et al. (2019) explored various unsupervised learning methods for online abnormality detection, focusing on feature selection and real-time detection capabilities using Isolation Forest and Lightweight Online Detector of Anomalies (LODA) [6].

Vafeiadis et al. (2019) utilized autoencoders to detect multiple types of errors across a large-scale smart meter network, enhancing maintenance automation and providing a practical tool for anomaly detection [7].

Khoshsefat et al. (2024) applied a CNN-based model for district-level anomaly detection in Tehran, focusing on distinguishing authorized from unauthorized usage. Their work highlighted the effectiveness of CNNs in localized detection contexts [8].

Singhal et al. (2022) presented an analytical study that provided a comprehensive evaluation of various machine learning techniques, emphasizing their effectiveness in managing electricity consumption and detecting anomalies such as energy theft and leakage. They explored different data analytics tools and proposed a systematic approach for anomaly detection using RNNs and LSTM networks, demonstrating high accuracy in identifying abnormal consumption patterns [9].

Himeur et al. (2021) offered a comprehensive review of Machine Learning techniques for anomaly detection in building energy systems. Their work highlighted the integration of non-intrusive load monitoring and contextual data, such as occupancy and ambient conditions, to enhance anomaly detection accuracy. They emphasized addressing challenges like the lack of annotated datasets, scalability, and real-time implementation, which align with future directions in the field [3].

Overall, these studies underscore the potential of combining multiple machine learning techniques to address the complexities of anomaly detection in energy systems. Future research could focus on integrating reinforcement learning and explainable AI, along with exploring transfer learning to extend the applicability of these models to diverse geographical regions and consumption patterns.

2.1.1.2 Future Directions and Trends

The reviewed literature suggests a growing trend towards hybrid models that combine multiple machine learning techniques to improve anomaly detection accuracy. There is also an increasing focus on integrating these models into energy management systems for real-time optimization and decision-making. Future research could explore the use of reinforcement learning and explainable AI to provide more transparent anomaly detection and optimize power usage in response to detected anomalies. Furthermore, the use of transfer learning and domain adaptation can enhance the applicability of these models across different geographical regions and contexts.

Chapter 3

Problem - Anomaly Detection in Smart Metres

3.1 Problem Definition

The aggressive deployment of smart meters across Ireland, as outlined in earlier chapters, has equipped the majority of households and businesses with the tools necessary for advanced energy management and billing accuracy. Despite these advancements, the full potential of this infrastructure is yet unmet due to persistent challenges in anomaly detection.

3.1.1 Current Scenario

Following the national rollout initiative, Ireland has committed to equipping every household with smart metering technology by 2024, a move driven by the goal to enhance energy efficiency and consumer engagement. However, this rapid expansion has introduced significant challenges in managing the voluminous data generated, particularly in detecting and addressing anomalies effectively.

3.1.2 Significance of Anomaly Detection

Anomalies in meter readings—ranging from minor inaccuracies to major discrepancies—can have far-reaching consequences. They affect not only billing and customer trust but also impede the overarching goal of energy conservation. Accurate anomaly detection is therefore crucial; it ensures that energy distribution is both efficient and reliable, aligning with national energy conservation goals.

3.1.3 Technical and Financial Impacts

The current anomaly detection rates are insufficient given the scale of smart meter implementation in Ireland. Improved detection capabilities are essential to reduce non-technical losses, which have historically accounted for a substantial percentage of energy wastage. Enhanced detection not only promises substantial cost reductions but also supports Ireland's commitments under various environmental accords to reduce energy waste.

3.1.4 Proposed Focus

This thesis proposes to tackle these challenges by leveraging the latest in machine learning algorithms and data analytics, as previously introduced in Chapter 2. By enhancing anomaly detection, the project aims to directly contribute to more accurate billing, reduced energy waste, and improved operational efficiencies within the national energy grid.

3.2 Objectives

The main objectives of this project are:

1. **Research and Develop an Anomaly Detection System:** Explore and evaluate machine learning-based methods for anomaly detection using Python libraries such as `scikit-learn`, `PyOD`, and `TensorFlow` to identify irregular consumption patterns. These libraries are selected as part of the research phase and represent potential tools rather than finalized choices.
2. **Reduce Detection Errors:** Investigate methods to incorporate contextual factors like operational hours and seasonal trends to reduce false positives by 20%, with an overall target of improving anomaly detection accuracy by up to 30%.
3. **Achieve a 25% Reduction in Non-Technical Anomalies:** Non-technical anomalies (NTAs) are consumption irregularities not caused by technical issues but by factors like unauthorized energy use (theft), faulty meter readings, or billing errors. This project aims to reduce NTAs by 25%, supporting Cork City Council's sustainability goals by identifying and mitigating such anomalies, thereby minimizing energy waste.

3.3 Functional Requirements

The functional requirements for this project have been defined as the following.

1. **Data Import and Storage:** Design a system to automatically import and store CSV data in a SQL database to facilitate time-series analysis.
 - Supports Objective 1: Research and Develop an Anomaly Detection System by providing a structured framework for data analysis.
2. **Anomaly Detection with Machine Learning (Planned):** Explore models such as Isolation Forest (from `scikit-learn`), LSTM (using `TensorFlow`), and Autoencoders (from `PyOD`) for detecting significant deviations in consumption data. These models are chosen based on their suitability for the task and will be evaluated further in this research phase.
 - Supports Objective 2: Reduce Detection Errors by improving accuracy and minimizing false positives.
 - Supports Objective 3: Achieve a 25% Reduction in Non-Technical Anomalies by leveraging advanced machine learning techniques.
3. **Context-Sensitive Reporting:** Develop a reporting framework that generates concise, context-aware anomaly reports, filtering out expected patterns (e.g., event-based spikes) to maintain report relevance and usability.
 - Supports Objective 2: Reduce Detection Errors by incorporating operational context to improve report relevance.
4. **View-Only Data Dashboard:** Provide a user interface to view reports, statistics, and analytics, including interactive graphs (line graphs, heatmaps) for analyzing consumption trends and detected anomalies.
 - Supports Objective 1: Research and Develop an Anomaly Detection System by facilitating insights into consumption patterns.
 - Supports Objective 2: Reduce Detection Errors by focusing on actionable notifications to reduce unnecessary alerts.

3.4 Non-Functional Requirements

The non-functional requirements for this project have been defined as the following.

1. **Scalability:** The database and system architecture should be designed to handle increasing data volumes as more smart meters are added across additional buildings, ensuring efficient anomaly detection as the dataset grows.
 - Supports Objective 1: Research and Develop an Anomaly Detection System by ensuring the system can accommodate future growth.
2. **Performance:** The system should perform anomaly detection on new data within each hourly reporting interval, enabling timely identification of irregularities in line with the reporting schedule.
 - Supports Objective 2: Reduce Detection Errors by ensuring real-time anomaly detection.
3. **Reliability:** The system should maintain a high level of reliability, with a target uptime of 99.9%, ensuring anomaly detection operates consistently without disruptions.
 - Supports Objective 1: Research and Develop an Anomaly Detection System by maintaining consistent functionality.
4. **Security:** All data related to anomaly detection should be encrypted at rest and in transit using industry-standard protocols. Access to the anomaly detection module should be restricted to authorized personnel.
 - Supports Objective 3: Achieve a 25% Reduction in Non-Technical Anomalies by safeguarding data integrity and trust.
5. **Maintainability:** The anomaly detection module should be implemented using a modular architecture to facilitate updates and the integration of new detection methods without disrupting existing functionality.
 - Supports Objective 1: Research and Develop an Anomaly Detection System by enabling future enhancements.
6. **Data Integrity:** Data validation checks should be applied to all input data points used in anomaly detection to ensure accuracy and consistency in detection results.
 - Supports Objective 2: Reduce Detection Errors by ensuring reliable and accurate data input.

3.5 Plan for Next Semester

To ensure the successful implementation of the project, a detailed timeline for the next semester has been created. The plan includes specific milestones, deliverables, and deadlines to maintain progress. Below is a Gantt chart outlining the key phases of the project:

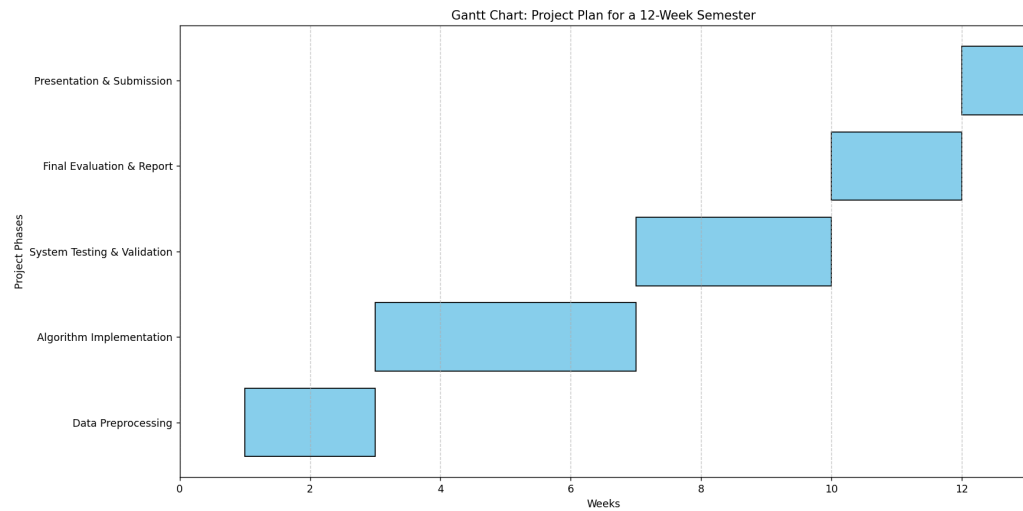


FIGURE 3.1: Gantt Chart: Project Plan for Next Semester

Key Phases of the Plan

1. Data Preprocessing and Cleaning (Week 1–2):

- Validate and preprocess data to ensure consistency and readiness for machine learning models.

2. Algorithm Implementation (Week 3–6):

- Develop and fine-tune machine learning models, including Isolation Forests and Autoencoders.
- Test statistical approaches like rolling averages and z-scores.

3. System Testing and Validation (Week 7–10):

- Evaluate model performance against known data.
- Optimize the system to minimize false positives while ensuring accurate anomaly detection.

4. Final Evaluation and Report Preparation (Week 11–12):

-
- Perform a final review of the system.
 - Document findings, including a comparative analysis of methodologies and results.

Chapter 4

Implementation Approach

4.1 Methodology

The methodology for this project is designed to ensure a structured and iterative approach to research, development, and evaluation. The project is divided into multiple phases, each targeting a specific aspect of the anomaly detection system.

4.1.0.1 Phase 1: Research and Requirement Gathering

During this phase, I conducted a comprehensive literature review to identify state-of-the-art techniques in anomaly detection and time-series data analysis. The primary goal was to select suitable algorithms (e.g., Isolation Forest, LSTM) and tools (e.g., Flask, Dash, MySQL) for the project. Additionally, I gathered the functional and non-functional requirements from the problem statement to guide the subsequent phases.

4.1.0.2 Phase 2: System Design and Planning

Based on the requirements, I developed the system architecture, highlighting the interactions between components such as data collection, storage, processing, and visualization. This phase also included creating a risk assessment matrix, defining evaluation metrics (e.g., precision, recall, runtime efficiency), and planning the implementation timeline.

4.1.0.3 Phase 3: Prototyping and Testing Plan

Although implementation will begin in February 2025, this phase involves planning for prototyping tasks. I created initial mock-ups for the dashboard interface using Dash

and React and outlined test cases for validating the anomaly detection algorithms. The CSV file is currently being used to simulate data input, which will transition to MySQL in the implementation phase.

4.1.0.4 Phase 4: Evaluation and Iteration Planning

I outlined an evaluation framework that includes unit testing for Python scripts, integration testing for system components, and runtime performance analysis. The evaluation metrics will focus on minimizing false positives, ensuring scalability, and maintaining user-friendly visualization. Iterative testing will ensure continuous refinement during the implementation phase.

This methodology ensures a research-driven, well-structured approach to building the anomaly detection system, with room for iterative improvements as challenges arise during implementation.

4.1.0.5 Research Method

The primary research for this project will draw from academic databases such as Google Scholar and IEEE Xplore, supplemented by other reputable online sources. The research approach will follow a systematic process to ensure comprehensive coverage of relevant topics:

1. Define and refine the research topic to establish a clear focus.
2. Gather background information from foundational articles and studies.
3. Utilize library catalogues to access books, media, and further resources on anomaly detection and machine learning.
4. Consult periodical indexes to find relevant research articles.
5. Supplement with internet resources, particularly for practical insights into the technologies involved.
6. Evaluate the reliability and relevance of all sources.
7. Properly cite all sources in a standardized format to maintain academic integrity.

This approach will ensure a thorough understanding of both theoretical and practical aspects of anomaly detection in energy data, as well as best practices in data management and machine learning model selection.

4.1.0.6 Developing Necessary Technical Skills

This project involves a diverse technology stack, including machine learning, database management, and frontend development. The following steps outline the approach for building these skills:

- **Machine Learning and Anomaly Detection:** The knowledge gained from machine learning and data analytics modules has provided a strong foundation in supervised and unsupervised learning methods, feature engineering, and data preprocessing. This project will build on that knowledge by exploring advanced anomaly detection methods through resources like StackOverflow and TensorFlow documentation. The focus will be on models such as Isolation Forests, Autoencoders, and LSTM networks, which are well-suited for time-series data analysis. Techniques learned in data analytics, such as data cleaning, normalization, and feature scaling, will also be applied to ensure high-quality inputs for model training.
- **React and Frontend Development:** Although frontend development is not the primary focus, familiarity with React will be essential for data visualization. Introductory tutorials from React's official documentation and platforms like Codecademy will be utilized to gain basic proficiency in React components, hooks, and API integration.
- **SQL Database Management:** Refresher tutorials on SQL and database management will be followed to handle time-series data efficiently, covering indexing, data retrieval, and management techniques to ensure optimized storage and querying.

This structured learning approach will allow for the gradual acquisition of necessary skills, with hands-on practice to reinforce knowledge as the project progresses.

4.1.0.7 Project Management Approach

For managing the project’s timeline and tasks, a hybrid approach called **Scrumban** (a combination of Scrum and Kanban) will be utilized. This method is well-suited to a flexible, research-driven project.

- **Kanban Structure:** Jira will serve as the primary project management tool, with columns like “To Do,” “In Progress,” “Testing,” and “Completed” for organizing tasks. Each task will be placed in a column reflecting its current status and moved accordingly as the project advances. This setup in Jira allows for clear tracking of progress and smooth transitions through each stage of the project.
- **Scrum Sprints:** The project will be divided into three-week sprints, with each sprint focused on completing a specific phase (e.g., literature review, model testing, or system integration). Sprints will help maintain steady progress while allowing flexibility to adapt as the research phase evolves.
- **Review and Adjustment:** At the end of each sprint, a review session will assess the progress, challenges encountered, and any necessary adjustments to the project plan. This iterative approach will ensure the project stays on track and can adapt to new findings or technical needs.

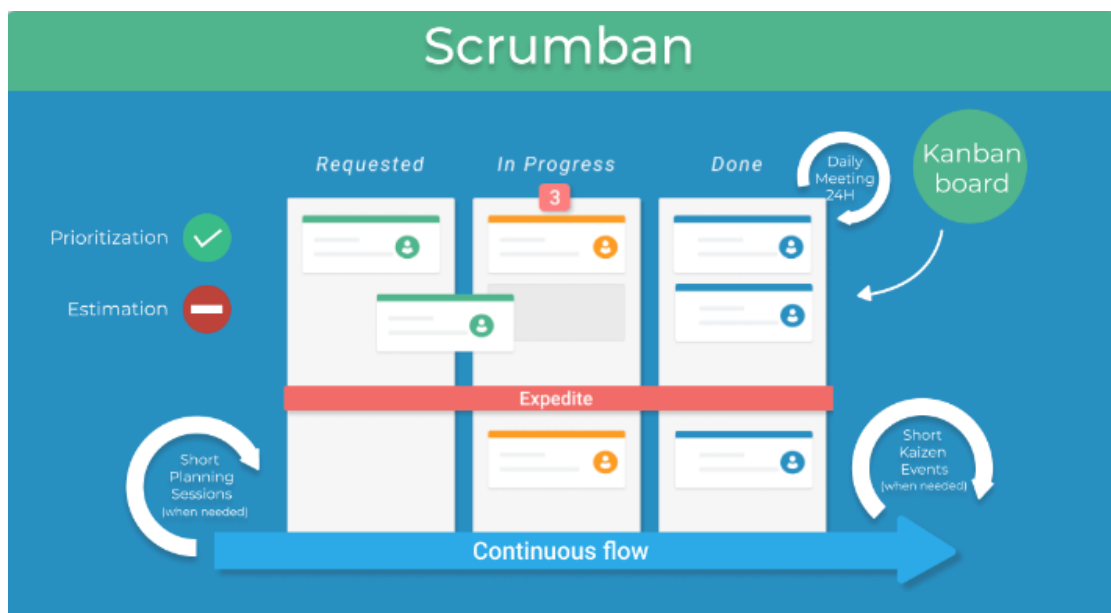


FIGURE 4.1: Scrumban Methodology [1]

By combining the task-based structure of Kanban with the iterative cycles of Scrum, this Scrumban approach will support both the exploratory and technical aspects of the project, ensuring that each phase progresses efficiently and adapts as needed.

4.2 Architecture

The architecture for this project involves several key technologies to support the anomaly detection system, data processing, and model training:

- **React** - Front-End Framework
- **Python** - Machine Learning and Data Processing
- **SQL Database** - Data Storage and Management
- **Visual Studio Code** - Integrated Development Environment

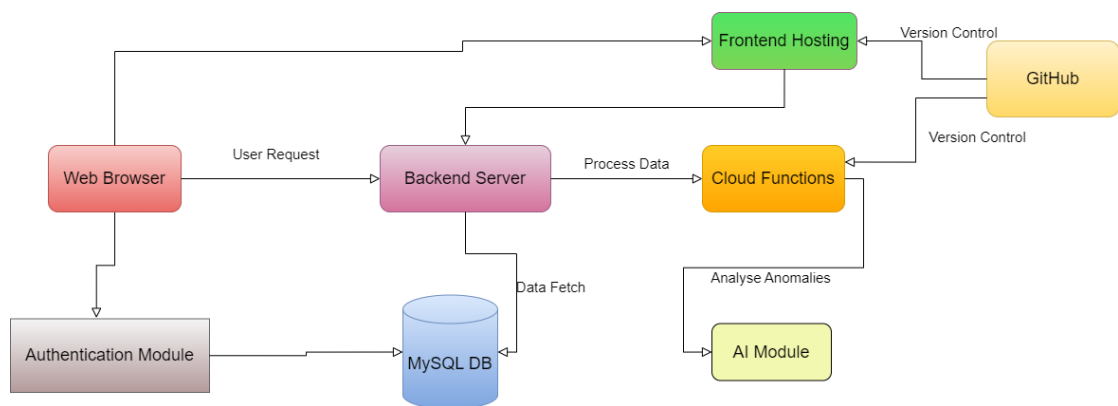


FIGURE 4.2: High Level Architecture Diagram for Web Application [2]

4.2.1 Technologies Involved

The system will rely on several core technologies:

Python: Python is the primary programming language for this project. It will handle data preprocessing and the implementation of machine learning algorithms such as Isolation Forest and Long Short-Term Memory (LSTM) networks. These models are crucial for detecting anomalies in the energy consumption data collected from the smart meters [10].

MySQL: For data storage, I'm using a MySQL relational database. This allows me to structure and organize the energy consumption data efficiently, keeping track of meter IDs, timestamps, and energy usage. It's reliable for storing both historical and real-time data, which the machine learning models will need for training and inference [11].

GitHub: Version control is a vital part of this project's development cycle. GitHub will help manage the codebase, track changes, and allow for collaboration during development. Using branches and commits, I'll be able to track progress and troubleshoot issues more easily [12].

JavaScript/React/HTML/CSS: I may use JavaScript along with HTML and CSS to create the front-end dashboard for the system. This dashboard will allow stakeholders to monitor energy data and view anomaly reports in real-time. The user interface will offer features such as live data visualization and the ability to configure alerts or thresholds for anomaly detection [13] [14].

I am planning to develop the front-end of this project using JavaScript, HTML, and CSS, with the possibility of using React depending on the needs that arise during development. There isn't a fixed, the structure outlined here is based on what I've used in the past. However, it could change as the project evolves during the next semester.

The project will likely have two main folders: `src` and `public`.

The `public` folder will include an `images` subfolder, which will store all the images used in the web application, such as icons and any visualizations related to energy consumption data.

The `src` folder will be organized into several subfolders:

Services Folder: This folder will contain the code necessary to interact with the back-end, including any APIs or database connections (like MySQL queries).

Pages Folder: This folder will hold the main pages of the application, such as the dashboard, anomaly reports, and user settings.

Components Folder: This folder will store reusable components, such as buttons, charts, and form elements that will be used across multiple pages in the application.

Context Folder: This folder will hold shared code that multiple components will need, such as API endpoints or system-wide notifications.

Styles Folder: This folder will contain all the CSS files used to style both the components and the pages, ensuring a consistent look and feel throughout the project.

4.2.2 Database Overview

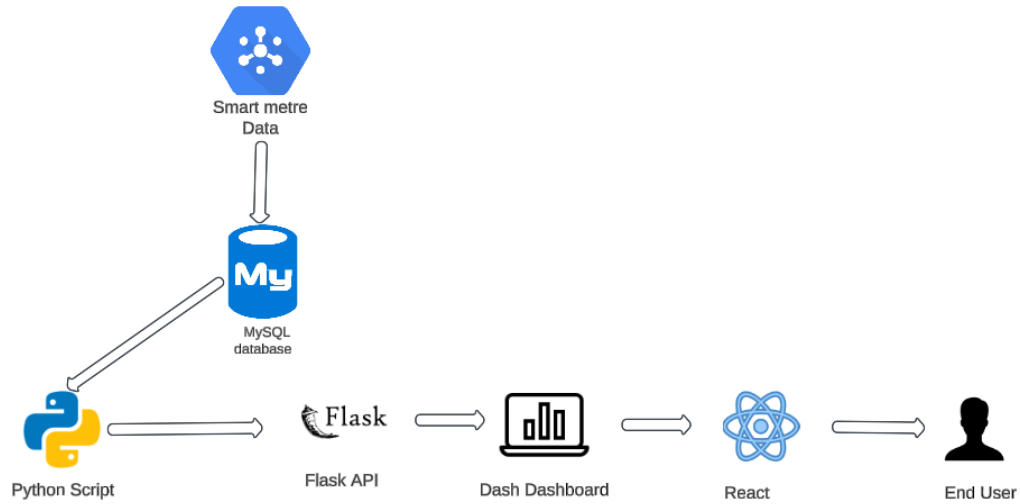


FIGURE 4.3: Complete System Architecture for Anomaly Detection System

4.2.3 Tools and Technologies

- **Visual Studio Code** - Visual Studio Code is a feature-rich and lightweight source code editor developed by Microsoft Corporation. It is used for writing and editing code for various parts of this project, including the *Python Scripts*, *Flask API*, and *React Frontend*. Features such as debugging, intelligent code completion, and embedded Git make it an excellent choice for development.
- **Flask** - Flask is a lightweight WSGI web application framework in Python. It is used to build the RESTful API that serves as the bridge between the *Python Scripts* for anomaly detection and the *Dash Dashboard*. Flask provides simplicity and flexibility, making it an ideal choice for prototyping and deployment on localhost.
- **Dash** - Dash is a Python framework for building web-based data visualization applications. It is used in this project to create an interactive *Dashboard* for visualizing gas consumption data and detected anomalies. Dash simplifies the process of combining data analytics and visualization in a single framework.
- **React** - React is a front-end JavaScript library used to build dynamic user interfaces. In this project, it is utilized for the *Frontend Application*, offering seamless interaction with the Dash app and delivering a responsive and engaging user experience.

- **MySQL** - MySQL is a widely used open-source relational database management system. It is employed to store and manage raw gas consumption data from smart meters, as well as processed data used for visualization.
- **Python** - Python is the core programming language used in this project. It powers the *data preprocessing*, *anomaly detection models*, and *Flask API*, enabling efficient analysis and seamless integration with other tools.

4.3 Risk Assessment

In the research phase of this anomaly detection project, several potential risks have been identified, along with strategies to mitigate their impact.

- **Risk 1: Data Quality Issues**
 - **Risk Category:** Frequent Major
 - **Explanation:** The accuracy of anomaly detection relies on high-quality, consistent data. Poor data quality could lead to inaccurate detections, resulting in false positives or missed anomalies.
 - **Mitigation:** Implement data validation checks during data ingestion to clean and standardize data, ensuring quality before analysis.
- **Risk 2: Model Overfitting and Performance**
 - **Risk Category:** Remote Critical
 - **Explanation:** Machine learning models could overfit if not properly tuned, causing inaccurate predictions on new data and reducing system reliability.
 - **Mitigation:** Use cross-validation and regularization techniques to improve generalisability and prevent overfitting.
- **Risk 3: High False Positive/Negative Rates**
 - **Risk Category:** Occasional Major
 - **Explanation:** Excessive false positives or negatives may decrease trust in the system, impacting its effectiveness in detecting true anomalies.
 - **Mitigation:** Continuously monitor model accuracy and adjust thresholds to balance sensitivity and specificity.

- **Risk 4: Data Processing and Utilization Challenges**

- **Risk Category:** Medium
- **Explanation:** Challenges in data processing could lead to inefficiencies and errors in anomaly detection.
- **Mitigation:** Develop robust data processing pipelines to ensure accurate and efficient data utilization.

- **Risk 5: System Integration Failures**

- **Risk Category:** Medium
- **Explanation:** Integration issues with existing systems could disrupt operational efficiency and delay project timelines.
- **Mitigation:** Pre-integration testing and phased roll-outs can help identify and mitigate integration challenges early.

- **Risk 6: Slow Model Loading or High Computational Overhead**

- **Risk Category:** Medium
- **Explanation:** The developed model may take too long to process or load large datasets, especially during real-time anomaly detection, leading to delays in identifying and reporting anomalies.
- **Mitigation:** Optimize the model's architecture and use efficient data structures. Implement preprocessing techniques to reduce computational overhead. Leverage university machines, cloud services like AWS or Google Colab, and parallel processing or hardware acceleration (e.g., GPUs) to handle computationally intensive tasks effectively.

- **Risk 7: Technology Obsolescence**

- **Risk Category:** Low
- **Explanation:** Rapid technological advancements may lead to the system becoming outdated, requiring updates or replacements.
- **Mitigation:** Keep the system architecture flexible and maintain up-to-date knowledge of technological trends to facilitate easy updates.

- **Risk 8: Unexpected Coursework Load**

- **Risk Category:** Medium
- **Explanation:** Increased coursework or academic responsibilities may reduce the time available to work on the project.

- **Mitigation:** Dedicate weekends to writing the report and use a detailed timeline to manage other tasks, ensuring steady progress despite coursework.

TABLE 4.1: Updated Risk Matrix

Frequency/ Consequence	1-Rare	2-Remote	3-Occasional	4-Probable	5-Frequent
4-Fatal					
3-Critical		Risk 2			
2-Major		Risk 6	Risk 3, Risk 5	Risk 7, Risk 8	Risk 1
1-Minor				Risk 4	

4.4 Implementation Plan Schedule

This implementation plan provides an outline of the remaining work to be completed by the end of Semester 2. The plan is designed to be ambitious but adaptable, ensuring that key milestones are met while allowing for adjustments as the project evolves.

4.4.1 Semester 1: Research and Initial Prototyping

- **October - November: Research and Background Development**

Complete the foundational research on anomaly detection methods and explore relevant Python libraries like `scikit-learn`, `PyOD`, and `TensorFlow`. Begin initial setup of the database structure to ensure smooth data ingestion for future stages.

- **November - December: Prototyping Core Detection Models**

Start prototyping with basic anomaly detection models, such as Isolation Forests and Autoencoders, to establish a working baseline. Initial tests will focus on understanding data behavior and identifying any preprocessing needs.

4.4.2 Semester 2: Full Development and Testing

- **January - February: Improving Model Accuracy**

Based on early prototyping results, refine the models and incorporate additional features to improve detection accuracy. Experiment with different model parameters and introduce LSTM models if time-series capabilities are beneficial.

- **March - April: Full System Integration and Testing**

Integrate the models with the database for real-time detection capabilities and

conduct thorough testing to validate the system's effectiveness. Adjust detection thresholds and reduce false positives through iterative testing.

- **April - May: Documentation and Presentation Preparation**

Finalize documentation for all project components, detailing system functionality and limitations. Prepare the project for submission, including a final presentation that demonstrates the system's key features and results.

4.5 Prototype

While no code has been implemented yet, I have developed a conceptual outline for the project's final design and functionality. This section provides a high-level vision of how the completed project might look based on initial planning and design considerations.

4.5.1 Envisioned Interface and Data Flow

The final interface will feature a user-friendly dashboard displaying key metrics, energy consumption trends, and detected anomalies. This dashboard would use Python's Plotly library or similar for interactive graphs, integrated within a simple web application to facilitate data visualization. The envisioned layout is focused on clarity, with sections for real-time data monitoring and historical anomaly trends.

4.5.2 Anomaly Detection Concept

The core of the project will involve implementing anomaly detection algorithms, such as Isolation Forests and Autoencoders, to identify irregularities in energy consumption patterns. This phase will use Python libraries like `scikit-learn` and `TensorFlow` to enable reliable detection. These models will be trained on historical data and configured to run on a local server for real-time analysis.

4.5.3 Deployment Plans

For initial deployment, the project is planned to run locally on a simple server setup, such as Flask, allowing the prototype to be accessible on a local network. This local setup will enable easy testing and debugging of the interface and backend processes. Should the project need to scale, future deployment to a cloud service like AWS could be explored.

4.5.4 Next Steps in Prototyping

Moving forward, the focus will be on initial implementation of the database structure, establishing the data ingestion process, and beginning basic model testing. These steps will build the foundation for a more complete, functional prototype in the next phase of development.

4.6 Implementation Progress

I have made initial progress in implementing the anomaly detection system for smart meters. Here are the key steps I have completed so far:

- **Set up MySQL Database:** I configured a MySQL relational database to store and manage the energy consumption data effectively.
- **GitHub Repository:** I created a GitHub repository to track code changes and maintain version control.
- **Jira Kanban Board:** I set up a Kanban board in Jira to organise tasks and monitor the project workflow.
- **Python Script for Anomaly Detection and Dashboard:** I developed a Python script that performs the following tasks:
 - **Preprocessing:** The script cleans and processes the raw data, sorts it by timestamp, and adds features like hour and day of the week. It also calculates changes in consumption and detects periods of zero consumption.
 - **Anomaly Detection:** Using rolling averages and standard deviations, the script identifies anomalies in energy consumption based on deviations from normal patterns.
 - **Dashboard Development:** I implemented an interactive dashboard using Dash and Plotly. The dashboard allows users to visualize energy consumption trends, detect anomalies, and view thresholds for normal usage.

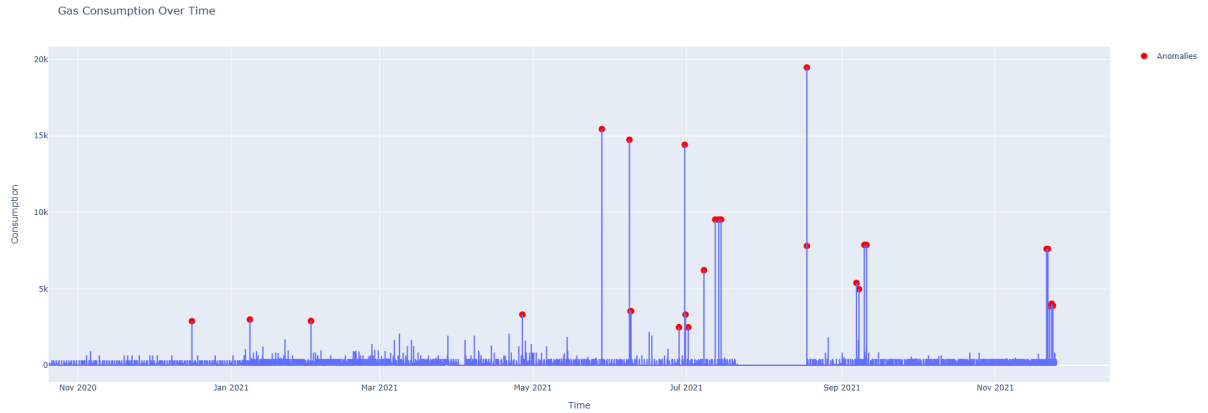


FIGURE 4.4: Interactive Dashboard Output

The dashboard output, as seen in Figure 4.4, highlights detected anomalies in gas consumption. However, there are several observations and critiques:

- **Spike Interpretation:** The anomalies identified as spikes could correspond to natural patterns, such as tanks running empty and being refilled. Without additional context, it is challenging to determine if these are true anomalies or expected behaviors.
- **Seasonality and Temperature Considerations:** Some anomalies occur during warmer months, which may not align with typical gas consumption trends. This suggests the model might need additional seasonal features to account for expected variations in usage.
- **False Positives vs. Missed Anomalies:** My focus on reducing false positives ensures the system maintains user trust by avoiding excessive alerts. While this approach occasionally misses anomalies, it is often preferable in practical scenarios to avoid overwhelming users with false alarms.
- **Feature Engineering Improvements:** Including domain-specific features such as tank refill schedules, weather data, or historical consumption trends could improve the model's ability to differentiate between true anomalies and expected patterns.
- **Data Labeling and Validation:** The model's performance would benefit from validation against labeled data (if available) to confirm that detected anomalies align with real-world events.
- **Visualization Enhancements:** Adding contextual layers such as thresholds, expected seasonal baselines, or annotations for known events would make the dashboard more interpretable for end-users.

Obviously, this model is not the most advanced or accurate. It is an initial implementation that will serve as a baseline for further improvement. I plan to refine the anomaly detection techniques, enhance the dashboard's functionality, and optimise the system's performance as the project progresses.

4.7 Evaluation

To measure the success of my project effectively, I will implement an evaluation plan that assesses both the functional requirements and the overarching goals outlined at the start. This evaluation will go beyond a binary “works/does not work” assessment by using specific metrics to quantify performance, accuracy, and reliability. The evaluation process will allow me to track progress and make adjustments as needed to ensure that the project meets its intended objectives.

4.7.1 Quantitative Metrics

The primary evaluation will focus on the following quantitative metrics:

- **Anomaly Detection Accuracy:** The core goal of the project is to develop a reliable anomaly detection system. I will evaluate the model’s accuracy in detecting true anomalies while minimizing false positives and false negatives. The target accuracy rate is above 85%, with separate metrics for precision and recall to provide a detailed view of model performance.
- **False Positive/Negative Rate:** This metric will measure the percentage of false alarms and missed detections. A high rate of false positives or false negatives would indicate an unreliable model, so I will aim to keep these rates under 10%.
- **Processing Time:** Since real-time detection is a goal, I will evaluate the time taken for each detection process. The target processing time for anomaly detection is set to under 10 seconds, ensuring the system provides timely results.

4.7.2 Qualitative Evaluation

In addition to quantitative metrics, qualitative evaluation will focus on user satisfaction and system usability:

- **User Feedback:** Feedback from initial testers or project supervisors will provide insights into the system’s usability and ease of interpreting results. Positive user experience is essential to ensure that the system can be used effectively in real-world settings.
- **Scalability and Maintainability:** The system’s architecture will be evaluated based on its ability to scale and adapt to future requirements. This includes assessing the modularity of code and the ease of adding new features without significant restructuring.

4.7.3 Comparison with Initial Goals

The final evaluation will compare the project's outcomes with the initial goals and functional requirements:

- **Meeting Functional Requirements:** Each functional requirement listed at the beginning of the project will be reviewed and tested individually to ensure completion. Any requirements that are not fully met will be documented, along with explanations and potential solutions.
- **Achievement of Initial Goals:** I will revisit the overall problem statement to assess whether the project has addressed the primary objectives effectively. This includes verifying that the system not only detects anomalies but also contributes to improved data quality and operational efficiency as intended.

4.7.4 4.8 Planned Implementation Details

The planned implementation of this anomaly detection system combines both straightforward baseline approaches and advanced machine learning techniques. This approach will allow me to create a solution that is both adaptable and effective in addressing Cork County Council's unique requirements.

4.7.4.1 4.8.1 Initial Testing Strategy

I will begin the implementation with a threshold-based system to establish baseline detection rules for specific patterns in gas consumption data. This approach provides a straightforward way to detect anomalies while gaining insights into usage trends:

- **Nighttime Usage:** I will flag gas usage exceeding 30% of typical daytime levels, as this could indicate potential leaks or accidental equipment usage.
- **Weekend Activity:** I will mark consumption above 40% of normal weekday usage, considering the reduced activity expected during weekends.
- **Sudden Spikes:** Any usage that is three times higher than normal will be flagged for immediate review.

While this strategy is simple, I anticipate challenges such as seasonal variations or false positives during colder weather. These limitations will guide the transition to more advanced methods.

4.7.4.2 4.8.2 Machine Learning Models for Advanced Detection

To enhance the robustness of the system, I plan to implement and evaluate several machine learning models for their suitability in detecting complex anomalies:

- **Isolation Forest:** I will deploy a forest of 100 trees with a contamination rate of 1%, leveraging its ability to handle high-dimensional data like smart meter readings. To ensure consistent results, I will apply a random state across experiments.
- **One-Class SVM:** Using the RBF kernel, this model will allow me to identify the boundary of normal data distributions. I will set $\nu = 0.05$ and optimise gamma values to balance false positives and negatives effectively.

- **Autoencoders:** These models will have symmetrical architectures to compress and reconstruct data. I will detect anomalies when reconstruction errors exceed a defined threshold, using mean squared error (MSE) as the loss function.

4.7.4.3 4.8.3 Statistical Methods for Complementary Analysis

I will complement the machine learning models with traditional statistical techniques to identify anomalies based on historical data trends:

- **Rolling Averages:** This method will help smooth out short-term fluctuations to identify broader patterns.
- **Z-Scores:** I will measure deviations from the mean to flag unusual readings.
- **3-Sigma Rule:** Any data point exceeding three standard deviations from the mean will be marked as a potential anomaly.

4.7.4.4 4.8.4 Implementation Framework

The system will integrate Python-based machine learning models for backend processing and React for the user interface. This architecture will allow me to create a scalable solution that remains user-friendly for non-technical staff. I will design the models to interface with Cork County Council's smart meter database, processing data in hourly intervals to ensure timely anomaly detection.

4.7.4.5 4.8.5 Scalability and Future Integration

To accommodate future growth, I am designing the architecture to be modular and scalable. This includes the potential for transitioning to cloud-based solutions, enabling centralized monitoring and improved computational efficiency as the number of monitored meters increases.

4.8 Conclusions and Future Work

4.9 Discussion

The research and planning phase of this anomaly detection project presented several significant challenges that have shaped both my approach and understanding of the

subject matter. One of the primary difficulties encountered was the complexity of technical research papers in the field of anomaly detection. While there are many literature papers, and my supervisor provided valuable guidance in selecting relevant papers, the mathematical complexities and the technical words of these papers often proved challenging. The advanced equations describing various machine learning models and statistical approaches required considerable effort to comprehend, particularly as they involved concepts beyond my current mathematical background.

To address this challenge, I adopted a learning approach, utilizing online resources including educational videos and supplementary materials to break down complex mathematical concepts into more manageable components. Despite these efforts, some of the more advanced mathematical frameworks remain partially understood, highlighting the depth and complexity of anomaly detection in smart meter systems.

The technical challenges went beyond the subject matter to include tools and processes. Learning to use LaTeX for document preparation introduced an additional learning curve. Initially, formatting errors were frequent and frustrating, but through utilizing the module's video resources and actually using it, I gradually developed proficiency with this essential writing tool.

Another significant challenge was managing the large dataset required for the project. Initially, my collaborator Alex attempted to send all the data via Gmail; however, due to Gmail's attachment size limits, the full dataset could not be delivered in one go. To resolve this, Alex split the data into eight separate CSV files, each containing 50,000 records. After receiving these files, I combined them into a single dataset. However, importing this large dataset into MySQL proved to be extremely time-consuming, with the process taking days to complete. To address this issue, I created a Python script that efficiently handles large datasets by processing and importing them in manageable chunks.

The script utilizes the pandas library to read the combined CSV file in smaller batches of 1,000 rows at a time, ensuring that memory constraints are not exceeded. It also processes the datetime column to match the required MySQL format and performs batch inserts using mysql-connector-python's executemany() function. This approach drastically reduced the time required to import the data, enabling me to complete the task efficiently without overloading the system.

Time management was also another significant challenge, particularly in balancing this research project with concurrent coursework assignments. I addressed this by establishing a structured approach, dedicating weekends to focused project work. This strategy,

while effective for maintaining progress, highlighted the importance of careful planning and resource allocation in academic research.

However, the most valuable lesson was learning to resist the urge to rush into implementation without an adequate theoretical foundation. The field of anomaly detection, with its numerous approaches and methodologies, initially felt overwhelming. This feeling of being overwhelmed gradually transformed into a deeper appreciation for the importance of thorough research and planning before implementation. Every moment of frustration became an opportunity to step back, reassess, and refine my approach, turning challenges into stepping stones for a more grounded and methodical strategy.

These challenges have significantly influenced my approach to the next phase of the project. By understanding the importance of balancing theoretical knowledge with practical implementation, I have shaped a project plan that is more realistic and well-grounded. Moving forward, I am determined to carry this balanced approach into the implementation phase, ensuring that technical decisions are firmly rooted in research findings and bolstered by the lessons learned so far. This phase marks the beginning of transforming theoretical efforts into tangible outcomes, building a system that reflects the depth of understanding gained through this journey.

The research and development of this anomaly detection system for Cork County Council's smart meters has provided several significant conclusions across different aspects of the project.

From the background research, it became clear to me that anomaly detection in smart meter data is an actively evolving field with multiple viable approaches. The literature review also shows a clear trend toward hybrid models that combine traditional statistical methods with advanced machine learning techniques. Particularly noteworthy was the success of combined approaches like those demonstrated by Lee et al. (2022) and Chahla et al. (2019), where multiple algorithms work together to achieve more reliable detection rates. This research also highlighted the importance of considering temporal patterns in energy consumption data, as demonstrated by successful implementations using LSTM networks and autoencoders. Additionally, it underscored the necessity of adapting methodologies to the specific operational and seasonal patterns unique to Cork County Council's dataset.

The problem definition phase showed me the complex nature of anomaly detection in the Irish context. Cork County Council's specific needs—monitoring 14 smart meters across various buildings with over 300,000 hourly readings—presented unique challenges that required careful consideration. The analysis revealed that effective anomaly detection

must balance technical accuracy with practical usability, especially considering the non-technical staff who will ultimately use the system. The problem is not merely one of statistical analysis but requires a comprehensive approach that considers operational patterns, seasonal changes, building-specific usage characteristics, and the Council's broader sustainability goals.

On the solution approach, several key conclusions were made: The use of a multilayered detection system that uses statistical methods in conjunction with machine learning algorithms gives the required flexibility to handle different anomaly types. The proposed architecture leverages Python for backend processing and React for the frontend interface, making it scalable and hopefully able to grow with the needs of the Council. The implementation strategy is to start with basic statistical methods and then move on to more complex algorithms. This allows me to make incremental improvements while maintaining system reliability. Furthermore, incorporating an iterative testing strategy ensures that the system evolves in line with both technological advances and organizational feedback.

The study also concluded that real-time processing capabilities, while desirable, had to be weighed against the need for accuracy and system stability. Locally implemented initial processing with future cloud deployment provides a practical path forward, which is consonant with both technical capabilities and organizational requirements. This approach also allows for future scalability and integration with additional meters or enhanced data streams, ensuring long-term relevance.

These conclusions give me a very strong backbone for the next phase of this project, where theoretical understanding shall be translated into a practical implementation system. The insights I will get from this research phase will go a long way to inform me in the development of such a system that is not only effectively anomaly detecting but also of value as a tool in improving energy efficiency in the operations of Cork County Council. Building on this foundation, I aim to create a system that seamlessly combines reliability, usability, and technical sophistication, ensuring it remains a valuable asset for years to come.

4.10 Future Work

This project has laid a strong foundation for anomaly detection smart meters, but there are several opportunities to expand and enhance its capabilities further. One key area is the integration of adaptive models that learn and change with new data, ensuring the system remains accurate as consumption patterns change over time. These models could automatically adjust to seasonal shifts and update real-time weather data more dynamically, further improving detection accuracy and drastically reducing any or all false positives.

Another promising direction is related to the development of predictive analytics. By transitioning from detecting anomalies to forecasting potential problems, the system could provide actionable insights to potentially avoid problems before they occur, enhancing energy efficiency and supporting long-term strategic planning for energy management.

It is also important to increase the system's scalability. While the number of monitored smart meters increases, transitioning to cloud-based processing could enable the system to handle larger datasets without compromising performance. A cloud infrastructure would also allow for easier centralized monitoring and integration with additional data streams, such as renewable energy sources.

Finally, it is very important to enhance the system's usability for non-technical staff. Introducing more intuitive dashboards, customizable alerts, and detailed explanations of the detected anomalies would make the tool more accessible and valuable for everyday operations. In this way, the system would stay robust but friendly while it develops to meet the further growth of the council's needs.

Chapter 5

Implementation

This chapter details the implementation of the anomaly detection system reflecting on the technical decisions made, the challenges encountered, and the improvements developed throughout the project. **The implementation work described here was completed during the second semester.** The work is critically assessed against the original design requirements, with particular attention given to computational efficiency, accuracy, limitations, and the originality of the approaches used.

Throughout the project, ethical considerations such as data privacy and responsible handling of consumption records were maintained. Although the smart meter data was anonymised, care was taken to ensure that no individual building or occupant could be personally identified. Future expansions of the system would continue to emphasise user privacy and secure data management practices.

The system evolved through continuous evaluation and refinement, balancing theoretical plans with practical challenges. Results, figures, and tables are referenced where appropriate to present a clear and coherent account of the implementation.

The primary objectives of the system were to accurately detect anomalies in gas consumption data, minimize false positives, and provide actionable insights for facilities managers. The system needed to balance the trade-off between precision and recall, prioritizing the reduction of false alarms over potentially missing some true anomalies, as specified in the thesis requirements.

The initial assessment of the system revealed several challenges that needed to be addressed:

- **Excessive false positives:** The system was generating an unusually high number of anomalies, often exceeding 100 per building, leading to a cluttered dashboard and reduced trust in the system's output.
- **Inconsistent evaluation metrics:** The evaluation function was reporting zeros for precision, recall, and F1 score, even when anomalies were being detected, indicating issues with the calculation and tracking of performance metrics.
- **Structural code issues:** Runtime errors suggested the presence of missing or improperly defined methods within the `EnhancedSmartMeterAnalytics` class, affecting the system's stability and reliability.

These challenges necessitated a thorough investigation and refinement process to improve the system's performance, usability, and alignment with the project's objectives.

5.1 Difficulties Encountered

This section categorises the internal and external difficulties encountered during the implementation phase of the anomaly detection system.

5.1.1 Internal Difficulties

Issue	Difficulty	Effect on Design
Data Quality Issues	Easy	Extra Preprocessing Required
Refill Pattern Detection	Easy	Redesign Anomaly Criteria
Real-time Visualisation Performance	Easy	UI Optimisation Required
False Positive Reduction	Hard	Changed Detection Approach
Anomaly Detection Logic	Hard	Too Many False Positives
Weather Data Integration	Medium	Needed Seasonal Adjustments
Machine Learning Model Performance	Hard	Memory and Scaling Issues
Context-Aware Detection	Medium	Multiple Threshold Baselines

TABLE 5.1: Summary of Internal Difficulties and Their Effects on System Design

5.1.2 Data Quality Issues

The gas consumption data collected from smart meters had significant issues, including missing values, anomalies, and inconsistencies, which made it hard to detect anomalies accurately. At first, I worked with CSV files, but as the dataset got bigger and I needed to do more database queries, I switched to using MySQL Workbench. When I was getting the data through Gmail, I ran into a problem. Because of file size limits, I could only get a quarter of the data at a time. To fix this, the solution was to send the data in multiple smaller files, making sure I got the complete dataset. When I tried to import the full dataset directly into MySQL Workbench, I found that the built-in import feature was very slow and often didn't respond. To solve this, I made a custom Python script to import the data in smaller, manageable chunks. This made the database import process much faster and more reliable. To fix the data quality issues, I had to do a lot of preprocessing, including:

- Filling in missing values using techniques like mean or median imputation
- Normalizing gas consumption values using techniques like min-max scaling.

By dealing with these data quality issues through preprocessing, I made sure that the data used in the anomaly detection models was reliable and accurate. This set the stage for effective anomaly detection.

5.1.3 Anomaly Detection Logic

The initial anomaly detection logic struggled to accurately differentiate true anomalies from regular usage variations, causing numerous incorrect anomaly flags. This challenge required a comprehensive redesign of the detection approach to improve the system's accuracy and effectiveness. At the beginning of the project, the anomaly detection logic relied on a simple threshold-based approach. It compared gas consumption values against fixed thresholds and flagged any deviations as anomalies. However, this simplistic method failed to account for the complex patterns and natural variations in gas consumption data, resulting in a high number of false positives as shown in Figure 5.1,. To address this issue, I started a full redesign of the anomaly detection logic. The improved approach incorporated advanced statistical techniques, machine learning algorithms, and domain knowledge to enhance the system's ability to identify genuine anomalies while minimizing false alarms. The redesigned anomaly detection logic included the following key components:

- **Statistical Analysis:** I used statistical methods such as interquartile range (IQR) analysis to identify outliers and unusual consumption patterns. These techniques helped establish dynamic thresholds that adapted to the inherent variability in the data as shown in Figure 6.4.
- **Machine Learning Algorithms:** Also I implemented advanced machine learning algorithms, specifically LSTM autoencoders and Isolation Forest models, to detect anomalies. These algorithms were trained on historical gas consumption data to learn normal patterns and identify deviations from those patterns.
- **Domain Knowledge Integration:** I then implemented domain-specific rules and expert knowledge into the anomaly detection logic. This included considering factors such as seasonal variations, holidays, and building occupancy patterns to adjust thresholds and improve the accuracy of anomaly detection as shown in Figure 6.4.
- **Contextual Anomaly Detection:** I also introduced contextual anomaly detection techniques to account for different operational contexts, such as weekdays, weekends, and holidays as one instance of such is shown at Figure 6.4. By considering when the gas consumption occurred, the system could more accurately distinguish normal changes from real anomalies.

5.1.4 False Positive Reduction

The early anomaly detection logic produced many false positives because the threshold criteria were too simple. As shown in Figure 5.1, these incorrect anomaly alerts reduced the system's effectiveness and reliability.

To fix this, a more detailed detection method was used, including:

- **Statistical Analysis and Dynamic Thresholds:** Used Python's Pandas and NumPy libraries to calculate rolling statistics, such as moving averages and standard deviations. This allowed the creation of dynamic thresholds that fit each meter's past energy use patterns, reducing false alarms by considering natural changes in energy usage.
- **Machine Learning Techniques:** Built Long Short-Term Memory (LSTM) autoencoders using TensorFlow and Keras to learn complex patterns in the data. Isolation Forest models from the scikit-learn library were also used for anomaly detection, effectively identifying unusual patterns compared to regular usage.
- **Time-Based Pattern Recognition:** Used time-based data (hour of day, day of week, seasonal adjustments) with Pandas to understand typical daily and weekly energy patterns. This helped the system better identify true anomalies that happened at unexpected times.
- **Expert Knowledge Integration:** Included specific rules based on expert knowledge from Cork County Council. These rules were written into Python scripts to improve detection accuracy further, especially for energy patterns related to known schedules and how buildings were used.

This detailed approach greatly reduced false positives, making the system more accurate and easier to use. It helped to highlight real anomalies quickly, allowing timely fixes and better building management.

However, finding the best balance between avoiding false alarms and catching real issues needed careful adjustment of settings, checks against historical data, and regular reviews of model performance.

In summary, these detailed technical improvements were essential for making the anomaly detection system reliable, helping to make clear, data-driven decisions and efficiently managing buildings.

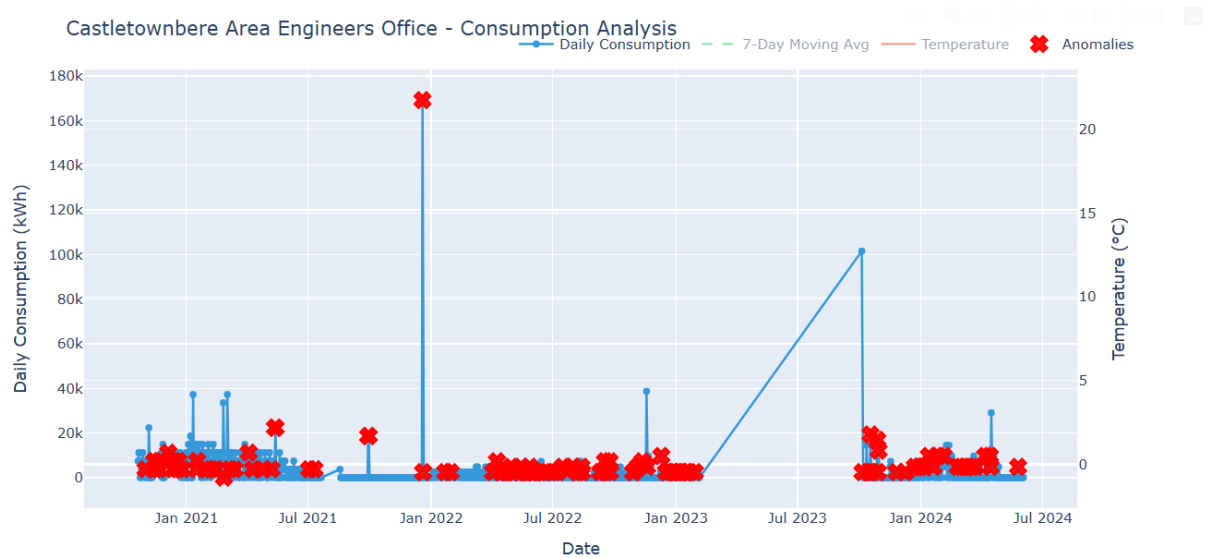


FIGURE 5.1: Example of Incorrect Anomaly Flags in Early Implementation

5.1.5 Refill Pattern Detection

Initially, the anomaly detection system struggled to accurately identify gas tank refill events, which often appeared as significant consumption anomalies. The challenge lay in distinguishing between genuine anomalies and normal refill patterns, which typically involve a sudden, substantial increase in consumption.

The refined detection approach moved beyond simplistic zero-consumption tracking to a more sophisticated statistical method. Instead of looking for consecutive zero days, the system implemented a nuanced algorithm that identifies refill events based on multiple statistical criteria:

- **Significant Consumption Increase:** A refill is detected when the current day's consumption is more than twice the previous day's consumption.
- **Absolute Consumption Threshold:** The absolute increase must exceed 20 kWh.
- **Statistical Deviation:** The consumption must be significantly higher than the recent median, determined using Median Absolute Deviation (MAD).

The detection logic is implemented through the following key steps:

1. Compare the current day's consumption with the previous day's consumption.

2. Check if the consumption increase meets the 2x multiplier criterion.
3. Verify the absolute increase is substantial (20 kWh).
4. Confirm the consumption is far from the recent statistical median.

This approach provides several advantages over simple zero-consumption tracking:

- More robust detection of refill events
- Reduced false positives
- Adaptability to varying consumption patterns
- Statistical rigor in identifying unusual consumption spikes

In the visualization, refill days are highlighted with distinctive markers, allowing users to quickly identify these important events in the consumption timeline. By accurately detecting and marking refill events, the system provides a more nuanced view of energy consumption patterns.

5.1.6 Machine Learning Model Performance

The advanced machine learning models, especially the LSTM Autoencoders and Isolation Forest algorithms, were really tough to work with when processing large datasets. They required a lot of computing power and memory, which made it hard to run them efficiently. On top of that, I had to learn how to build these models from scratch, which added to the difficulty and time pressure of the development process. It wasn't easy to get these complex models up and running smoothly. To handle these challenges, I had to find ways to make the training process more memory-efficient. This involved techniques like processing the data in smaller batches rather than all at once. I also had to be really careful about handling errors that might come up during the training process. Another thing I did was to optimize the code for the models, trying to make it run as fast as possible while still getting good results. This often meant finding a balance between model performance and computational efficiency. Despite these efforts, the performance of the models was still somewhat limited by the available computing resources and the size of the datasets. I had to work within these constraints and find ways to get the best possible results.

5.1.7 Weather Data Integration

Initially, effectively integrating weather data to account for temperature-driven variations in gas consumption was problematic. My supervisor recommended using the dataset available at www.degreedays.net; however, it turned out to be a paid service. As a result, I sourced weather data from an alternative free website www.yr.no. The solution involved developing an analytical framework that grouped data by seasonal and temperature ranges, allowing the establishment of more accurate, temperature-specific thresholds and significantly reducing false anomalies during regular weather-driven consumption fluctuations. As shown in Figure 6.9.

5.2 Real-time Visualisation Performance

The dashboard initially faced performance issues when dealing with the large dataset of 400,000 meter readings in real-time. The size of the data caused the dashboard to struggle with processing and displaying information quickly, leading to slow loading times and a poor user experience. To troubleshoot and test the visuals and results, I had to limit the querying to just 5,000 records from the dataset, as processing the entire dataset was extremely time-consuming. Moreover, as shown in Figure 5.2, the dashboard encountered frequent callback errors, which further hindered its functionality and usability. These errors occurred when interacting with specific data points or selecting certain options, causing the dashboard to become unresponsive or display incorrect information. The callback errors were a significant challenge that needed to be addressed to ensure a smooth and reliable user experience. After discussing these challenges with my supervisor, we decided to utilize Google Colab as a solution. By leveraging the computational resources provided by Google Colab, I was able to significantly improve the compilation time and overall performance of the dashboard.

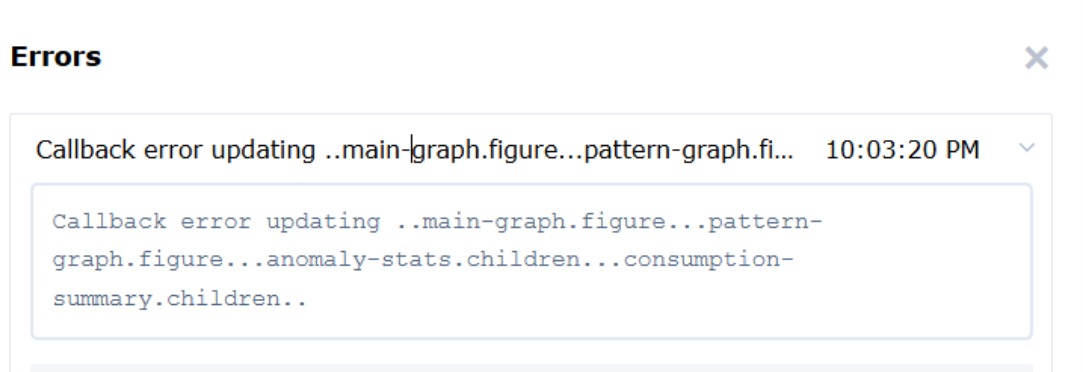


FIGURE 5.2: Example of the constant call back errors

5.2.1 Context-Aware Detection

Initially, the anomaly detection system applied the same thresholds and rules across all time periods, lacking the ability to differentiate between weekdays, weekends, working hours, and non-working hours. as shown in Figure 5.1. This led to a high number of false positives.

To address this, I implemented context-aware detection techniques in the `add_domain_knowledge_rules` method. The enhanced system now considers:

- Day of the week
- Holiday status

Rules by building:

- **Office buildings:** Detects weekend anomalies based on historical weekend averages and standard deviations.
- **Libraries:** Analyzes daily patterns; flags deviations considering weekday/weekend differences.
- **Skibbereen Library:** Custom rules for Mondays (closed); detects high usage on closed days and low usage on open days. as shown in Figure 6.4,
- **Fire stations:** Defines min/max usage thresholds using historical percentiles.
- **Community centres:** Uses rolling median and MAD to detect outliers amidst variable usage.

This reduced false positives and improved detection accuracy. Implementation required collaboration with domain experts and iterative refinement of thresholds per building type.

5.3 External Difficulties

Issue	Difficulty	Effect on Project
Course Workload	Medium	Reduced Available Focus and Time
Ramadan	Hard	Limited Energy and Disrupted Routine
Sickness	Hard	Lost Development Time and Momentum

TABLE 5.2: Summary of External Difficulties and Their Effects on Project Progress

5.3.1 Course Workload

The heavy workload from concurrent coursework significantly impacted my ability to maintain consistent focus and allocate adequate time for this project. Balancing multiple deadlines, assignments, and exams from other modules restricted the dedicated time available for detailed development and thorough testing phases. To manage this, careful prioritisation and strict time management strategies were adopted, although it still somewhat reduced my efficiency and the overall pace of development.

5.3.2 Ramadan

Observing Ramadan presented substantial challenges, primarily due to limited energy levels during daylight hours and disrupted routines. The fasting schedule reduced productivity and impacted my ability to maintain sustained periods of high concentration, particularly affecting complex and cognitively demanding tasks like debugging and algorithm fine-tuning. I adjusted my working hours to maximise productivity during periods when my energy levels were comparatively higher, though this inevitably slowed overall progress.

5.3.3 Sickness

Illness significantly disrupted my development schedule and momentum. I was diagnosed with a very bad bacterial infection, my physical condition considerably impaired my ability to work effectively, leading to delays in critical project milestones. Recovery required additional rest periods, creating gaps in continuity and hampering workflow consistency. To mitigate this, upon recovery, I dedicated extra time to catch up and re-organise tasks, although the interruption still resulted in noticeable delays and decreased project efficiency.

5.4 Actual Solution Approach

Chapter 4 presented the theoretical solution based on architectural design, proposed use cases, risk analysis, development methodology, an implementation timeline, evaluation criteria, and an initial prototype. Between January and April, this proposed approach evolved into a complete, functioning system. This section compares the originally proposed solution with the actual developed system across each key area. Any changes made are explained and justified in light of implementation realities.

Final Architecture

The implementation of this system followed the core architecture I outlined in Chapter 4 — with Python handling the backend, MySQL managing the data, and Flask acting as the API bridge to the frontend. Originally, I planned to build the front-end using React. However, after discussions with my supervisor, it was decided to use Angular instead. The main reason for this change was to align better with the existing platform where the system will be launched. The good news is that the Dash visualisations didn't need to change — they still run through Flask and are compatible with the Angular frontend. So while the frontend stack changed, the overall system functionality has remained consistent with my original design and is now better aligned with the environment it is intended to be deployed in.

Use Case Comparison

The following use cases were originally planned and were successfully implemented:

- Aggregation of raw smart meter readings into daily totals.
- Extraction of advanced features including rolling statistics, seasonality, refill patterns, and weather correlation.
- Application of two anomaly detection models: Isolation Forest and LSTM Autoencoder.
- Logging of anomalies to the MySQL database with severity scoring, [see Figure 6.4](#).
- Generation of natural language explanations for each detected anomaly, [see Figure 6.8](#).
- Visualisation of consumption trends and anomalies in a dynamic dashboard, [see Figure 6.8](#).

Additional features not present in the original plan were also added:

- A refined refill cycle tracker using MAD-based logic.
- Dynamic building-type rules to enhance context-aware detection (e.g., office vs fire station).

Risk Assessment Reflection

The original risk assessment correctly anticipated major implementation challenges. Key risks and how they were handled include:

- **False Positives:** Addressed using multi-layer verification logic and persistence filtering.
- **Refill Pattern Misclassification:** Managed by tracking refill cycles and suppressing false spikes through contextual awareness.
- **Data Quality Issues:** Handled by carefully preparing data and building flexible detection methods that work across different meter types.
- **Weather Context Integration:** Implemented using a simplified regression-based comparison for temperature shifts [see Figure 6.9](#).

No entirely new risks emerged; however, the time required to tune rule-based logic and handle refill-specific edge cases was underestimated.

Methodology Reflection

The development followed a structured Scrumban approach via a Trello board.

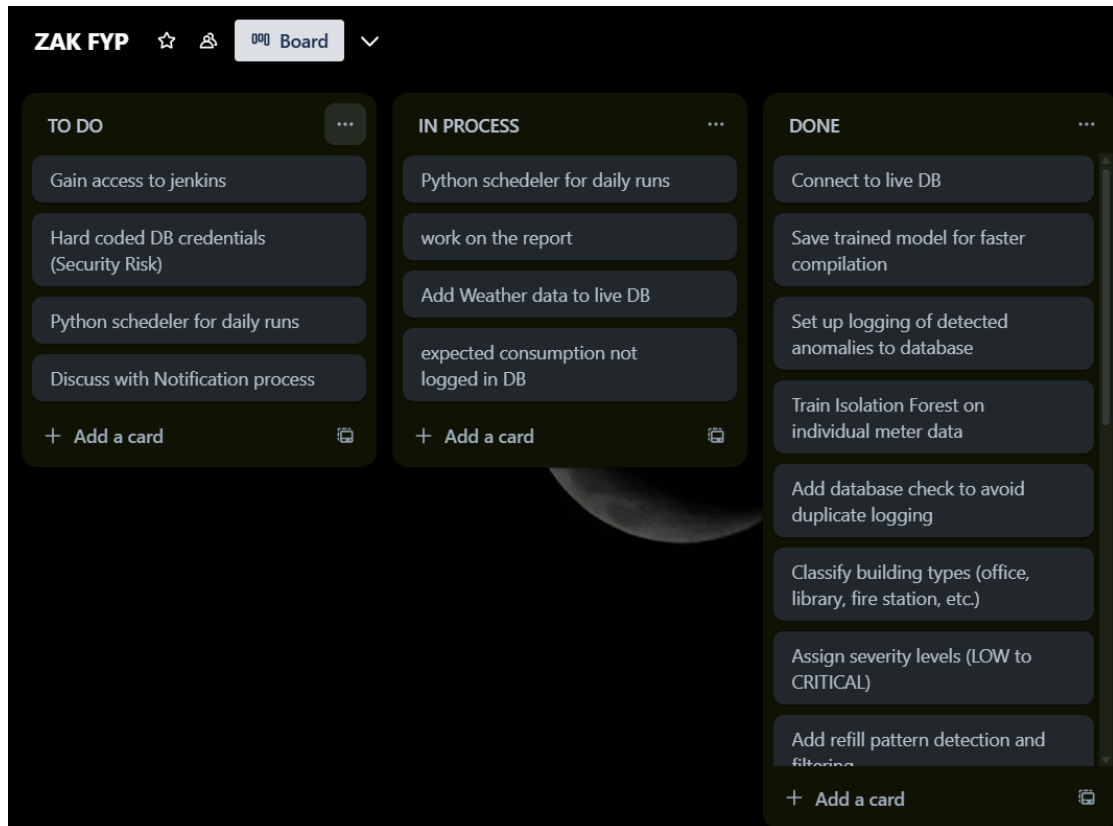


FIGURE 5.3: Trello board showing task progression across sprints using a Scrumban approach.

Sprint Breakdown and Schedule

The timeline from January to April was structured into the following sprints. The use of a Trello board to manage sprint tasks and visualise progress is shown in Figure 5.3.

Sprint One – Environment Setup and Initial Exploration

Sprint one focused on laying the groundwork for the system. The first task was connecting Python to the MySQL database and validating that data could be queried and processed correctly. Basic scripts were written to load and clean the raw smart meter data, and tables were created to store daily aggregated readings.

Once the setup was complete, initial data exploration was carried out. Simple visualisations were generated to observe trends over time. as shown in Figure 4.4.

This sprint was mainly about setting up the environment and confirming that the data pipeline worked as intended, which helped shape the direction for future development. No major issues occurred.

Sprint Two – Feature Engineering and Refill Logic

In sprint two, the focus shifted towards extracting meaningful features from the gas consumption data. Statistical features such as rolling mean, standard deviation, median, and median absolute deviation (MAD) were created to better capture fluctuations in usage.

A refill detection mechanism was also implemented to identify sharp increases in gas levels, which typically signal refills. This was followed by logic to track refill cycles, allowing the system to differentiate between regular consumption and refill-related spikes.

To improve anomaly detection accuracy, seasonal and calendar-based awareness was added. This included flagging weekends, and holidays, which typically show different consumption behaviours. as shown in Figure 6.6. These contextual indicators were prepared for integration into the model during future sprints.

Sprint Three – Model Development

Sprint three focused on building and testing the anomaly detection models. Both Isolation Forest and LSTM Autoencoder models were trained using the engineered features and historical gas consumption data. Each model was evaluated for its ability to capture irregular patterns across different buildings.

To support scalability and performance, model saving and loading were implemented on a per-meter basis. This allowed for efficient reuse of trained models without retraining each time the system restarted see Figure 6.12

The developed models use different ways of looking at the data, helping us understand how energy is used in different types of buildings.

Sprint Four – Anomaly Detection and Rule Logic

Sprint four focused on refining how anomalies were identified and scored. A combined scoring logic was developed to incorporate results from the Isolation Forest, LSTM Autoencoder. This allowed the system to make more reliable decisions by considering multiple detection sources.

To reduce false positives, persistence filtering was introduced. This meant that short-lived or one-off spikes would be ignored unless they persisted over a certain period, helping to distinguish between noise and genuinely abnormal behaviour.

Finally, building-type specific rules were integrated. These rules accounted for expected consumption patterns unique to libraries, offices, or community centres, and helped the system make more context-aware decisions when flagging anomalies [see Figure 6.4](#).

Sprint Five – Dashboard and Finalisation

Sprint five focused on integrating the system components and preparing for deployment. The Flask API was finalised to serve the processed anomaly data and support interaction with the Dash visualisation components. These Dash elements were embedded and connected to the anomaly detection output, allowing anomalies to be clearly overlaid on building-level graphs.

As per discussions with my supervisor, the frontend framework was switched from React to Angular to match the platform where the system will be deployed. Natural language explanations is generated for each flagged anomaly, alongside severity labels to help prioritise responses.

To support deployment, the system integrates with the existing internal platform using Jenkins, automating the build and deployment process. Additionally, live weather data is sourced to enhance anomaly context, especially for temperature-sensitive consumption patterns, [A clear view of the overall system flow in shown at Figure 6.1](#).

Evaluation Strategy Comparison

The original evaluation strategy focused primarily on pattern-based testing and manual validation of known anomaly scenarios. However, as development progressed, it became clear that a more nuanced approach was necessary to reflect the operational complexity of real-world buildings.

The evaluation process was expanded to include:

- Manual review of flagged anomalies against known refill patterns to validate detection logic. [as shown in Figure 6.11](#).
- Sensitivity testing using meters with predictable behaviour, such as offices that show no weekend usage, to check for false positives.
- Inspection of severity scoring to ensure high-impact anomalies is being correctly prioritised and not overlooked. [as shown in Figure 6.10](#).

These additions allowed for a more realistic assessment of the system's effectiveness and improved trust in its outputs. They also highlighted the importance of context-aware validation, especially when dealing with edge cases and non-standard usage patterns.

These additions improved the interpretability and practical usability of the model in a real-world context.

Prototype vs Final System

The original prototype was built around a basic Isolation Forest model applied to a small, static CSV dataset. Visual outputs were generated in Jupyter Notebooks, and minimal automation was present. The goal at that stage was to prove that anomalies could be detected in principle.

In contrast, the final system has evolved into a more complete and research-driven implementation with multiple enhancements:

- Automated daily data ingestion and feature extraction, removing the need for manual processing [see Figure 6.12](#).
- Integration of two complementary models (Isolation Forest and LSTM Autoencoder) for improved detection reliability.
- A rules engine and severity labelling system to help prioritise responses based on impact [see Figure 6.10](#).

- A live, interactive Angular dashboard embedded via Flask and Dash, providing natural language insights and anomaly overlays.

This progression represents a significant shift from a proof-of-concept to a robust, extensible system. While the original architecture laid a strong foundation, the final implementation reflects practical adjustments based on feedback, domain knowledge, and deployment needs.

The final system significantly extends the scope, automation, and robustness of the original prototype while still remaining aligned with the original vision outlined in Chapter 4.

Chapter 6

Testing and Evaluation

This chapter presents a structured evaluation of the anomaly detection system. The primary focus is on quantitative analysis, assessing the system's performance using real-world smart meter data from Cork County Council. Multiple evaluation metrics are used to objectively measure how accurately the models detect anomalies.

The chapter covers the following:

- The metrics used to evaluate anomaly detection effectiveness.
- The testing setup, including datasets, tools, and experiment design.
- A results section comparing different models, including LSTM autoencoders, Isolation Forest, and domain rule-based detection.
- A comparative analysis against standard baseline methods.
- Identified threats to the validity of results, *e.g.*, false positives.

The goal is to determine how well the system achieves our intended outcome of reliably detecting irregular gas consumption patterns in smart meter data.

6.1 System Testing

The anomaly detection system was tested using live smart meter gas consumption data collected from 14 Cork County Council buildings over a three-year period. The data was stored in a MySQL database and accessed using Python script. The objective was to evaluate the performance of the system across different detection methods: LSTM Autoencoder, Isolation Forest, and domain-specific rule-based detection.

6.1.1 Testing Environment

- **Hardware:** Huawei laptop, Intel Core i5 10th Gen CPU, 16GB RAM
- **Software Stack:** Python 3.10, TensorFlow, Scikit-learn, Dask, MySQL 8, Dash (Plotly)
- **Database:** `elighthouse_building`, hosted on a remote server, storing historical data readings

6.1.2 System Flow Overview

Before evaluating the anomaly detection models, it is important to understand the overall flow of the system from data processing to deployment. The following diagrams provide a high-level view of how different components interact within the solution.

Figure 6.1 presents the sequential steps followed during system operation. It illustrates how data flows through preprocessing, anomaly detection, and then into the dashboard for visualisation and alert generation.

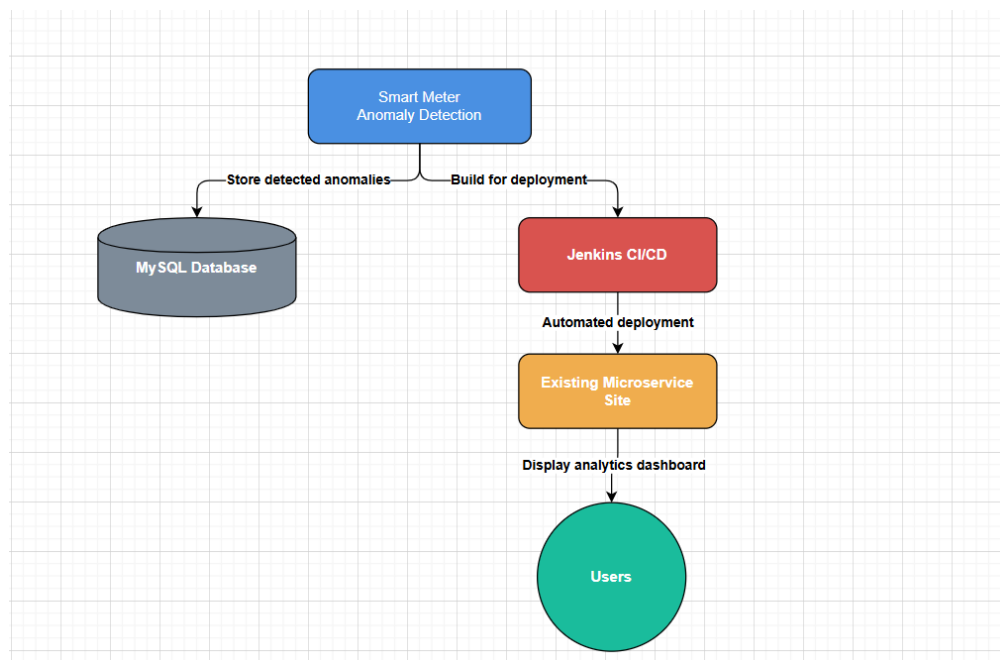


FIGURE 6.1: Flowchart showing deployment and integration of the anomaly detection system.

Figure 6.2 complements this by showing how key components like MySQL, the anomaly detection engine, weather integration, and the Dash dashboard are connected. It highlights the real-time interaction between data sources, processing modules, and the user interface.

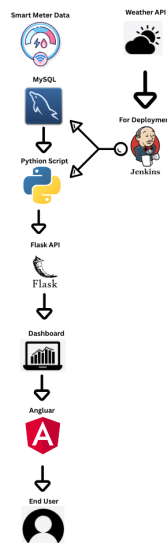


FIGURE 6.2: Overall architecture of the Smart Meter Anomaly Detection System showing component interactions and data sources

6.1.3 Experimental Design

The data was preprocessed and enriched with additional features, including rolling statistics, day-of-week indicators, holiday flags, and temperature variations. Refills were detected using robust statistical logic to ensure these points were not flagged as anomalies see Figure 6.11.

Each detection method was tested independently on a per-building basis:

- **LSTM Autoencoder:** Trained using 14-day sliding sequences. The model reconstructs expected patterns and flags anomalies based on high reconstruction errors.
- **Isolation Forest:** Trained using extracted numerical features. Points identified as isolated were marked as potential anomalies.
- **Domain-Specific Rules:** Applied to detect patterns such as weekend spikes in office buildings or unusual usage in libraries on days they are expected to be closed.

Anomalies detected were then logged into the `smart_meter_alarm` table along with contextual explanations, severity levels, and confidence scores see Figure 6.13.

6.1.4 Testing Objectives

The following aspects were evaluated:

- Ability to detect true anomalies (consumption irregularities, faulty readings, unexpected usage)
- Reduction in false positives through rule filtering and persistence checks
- Comparison of model output against known building schedules and historical usage
- Explanation generation quality for flagged anomalies see Figure 6.10

6.1.5 Visual Results

Below shows the system's data preparation process, showing the aggregation of consumption data, feature extraction, and threshold calculation across all meters.

```

Fetching daily aggregated data...
Query executed, got 18914 rows
Unique meter IDs: ['4663' '4665' '4664' '4832' '4839' '4840' '4837' '4841' '4842' '4859'
'4833' '4836' '4834' '4835']
Date range: 2020-10-20 00:00:00 to 2025-04-26 00:00:00
Starting enhanced anomaly detection...
Extracting enhanced features for improved anomaly detection...
Calculating rolling statistics...
Calculating building-specific thresholds...
Building-specific thresholds: {'4663': 3.000000000000001, '4665': 4.8000000000000025, '4664': 4.8000000000000025, '4832': 4.800000000000000
25, '4839': 4.6000000000000002, '4840': 4.8000000000000025, '4837': 4.6000000000000002, '4841': 4.8000000000000025, '4842': 4.800000000000000
25, '4859': 4.8000000000000025, '4833': 4.2000000000000002, '4836': 4.4000000000000002, '4834': 3.6000000000000014, '4835': 4.600000000000000
2}
Processing 14 meters...

```

FIGURE 6.3: Console output showing data aggregation, feature extraction, and threshold calculation during system preparation for testing.

Domain knowledge rules helped explain anomalies clearly by showing what went wrong, possible reasons, and suggested actions. In this example, the library showed high usage on a Monday when it was supposed to be closed.

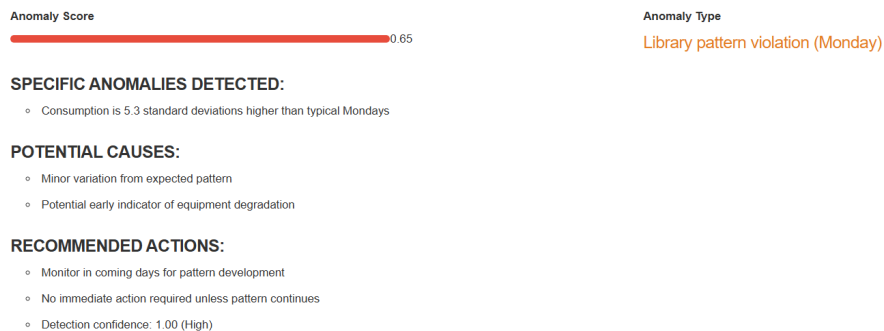


FIGURE 6.4: Example of a detected anomaly using domain knowledge rules.

To make the results easy to understand, all outputs were rendered on a real-time dashboard built with Dash and Plotly. Anomalies were visualised per building with overlays of actual and expected usage, with the real anomalies detected being shown as red X's.

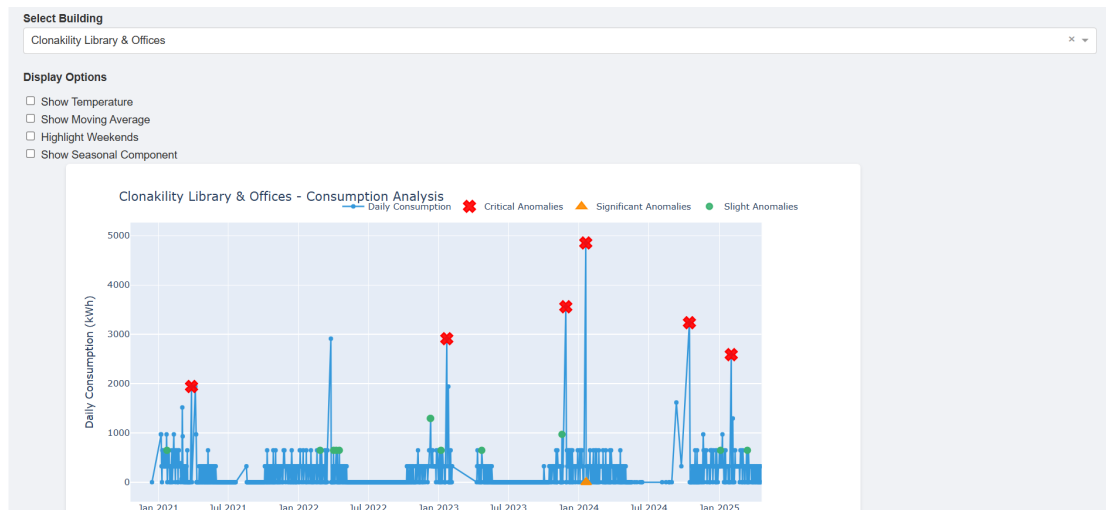


FIGURE 6.5: Dashboard view of real anomalies detected)

The Dash interface also includes controls to highlight specific data segments, such as weekends or holidays. This helps in quickly identifying patterns or outliers that align with operational cycles.

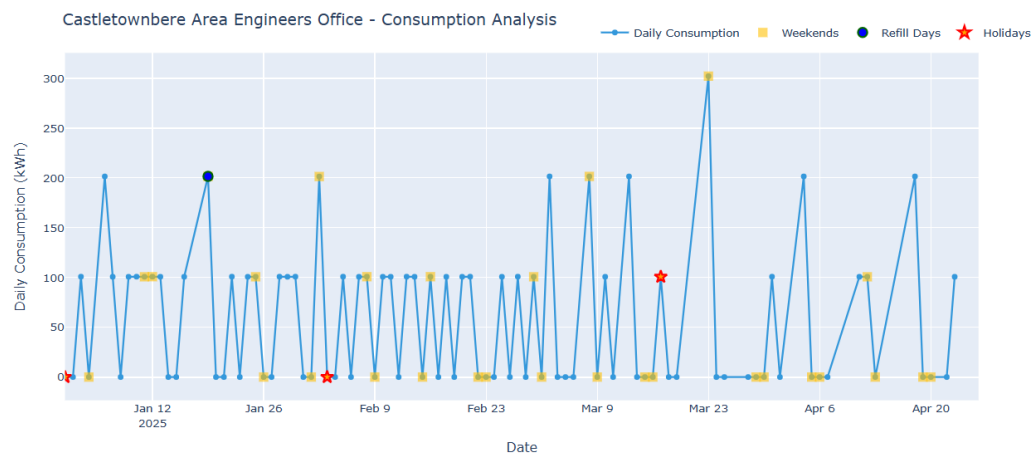


FIGURE 6.6: Interactive visual with weekend highlighting

To complement anomaly detection, general consumption patterns were visualised. These provided insights into typical usage trends across days of the week and months of the year.

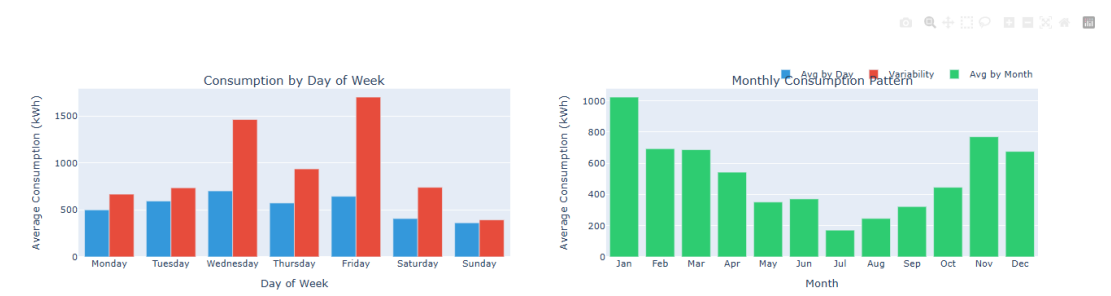


FIGURE 6.7: Average consumption by day of week and by month

The dashboard also provides a detailed breakdown of anomalies, including anomaly types, actual vs expected consumption, severity scores, possible explanations and what actions for administrator(s) to take.

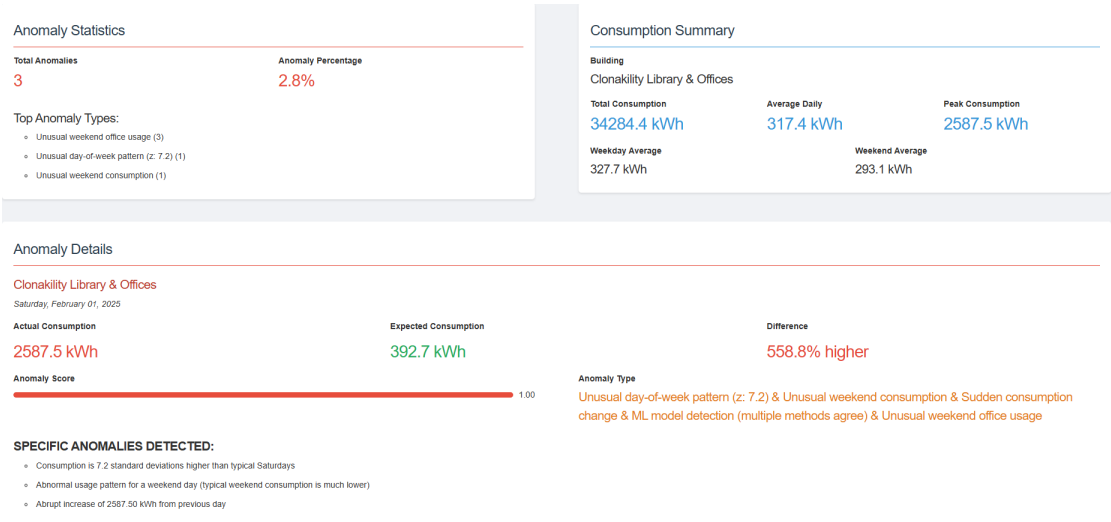


FIGURE 6.8: Detailed anomaly explanation.

6.1.6 Incorporating Weather Context

To provide additional context for anomaly detection, daily temperature data was included alongside gas consumption. Figure 6.9 shows the relationship between daily consumption (blue line) and temperature (red line). This allowed better interpretation of consumption patterns that may be influenced by weather changes.

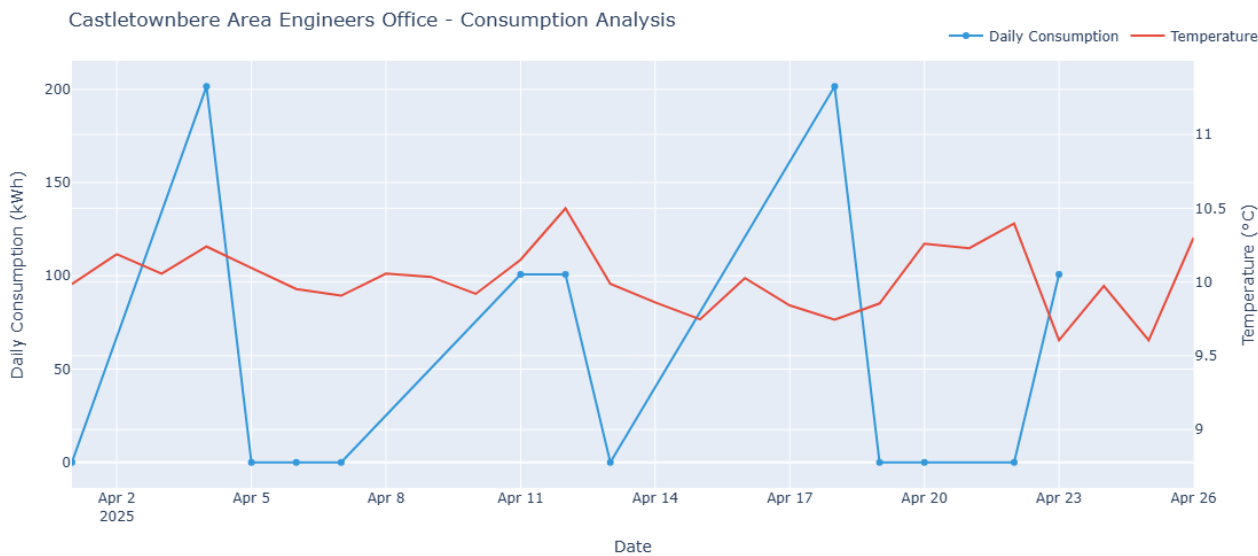


FIGURE 6.9: Daily gas consumption and corresponding temperature for Castletownbere Area Engineers Office.

6.1.7 Anomaly Severity Levels Visualization

Below highlights how the system distinguishes between different levels of anomalies, such as Critical, Significant, and Slight. Each level comes with its own potential causes and recommended actions, helping to prioritise responses based on the severity and confidence of detection.

POTENTIAL CAUSES: ◦ Heating/cooling system malfunction ◦ Major deviation from scheduled operation RECOMMENDED ACTIONS: ◦ Schedule inspection within 48 hours ◦ Review building operation schedule and recent changes ◦ Check HVAC control settings and operation ◦ Detection confidence: 0.80 (High) SPECIFIC ANOMALIES DETECTED: ◦ Abrupt decrease of 15122.59 kWh from previous day POTENTIAL CAUSES: ◦ Equipment left running outside normal hours ◦ Inefficient operation of heating/cooling systems ◦ Change in building usage not reflected in baseline RECOMMENDED ACTIONS: ◦ Review building schedule and occupancy patterns ◦ Check timer settings on major equipment ◦ Inspect for equipment running outside needed hours ◦ Detection confidence: 0.95 (High)	CRITICAL	POTENTIAL CAUSES: ◦ Minor variation from expected pattern ◦ Potential early indicator of equipment degradation RECOMMENDED ACTIONS: ◦ Monitor in coming days for pattern development ◦ No immediate action required unless pattern continues ◦ Detection confidence: 0.80 (High)	Slight
--	-----------------	--	---------------

FIGURE 6.10: severity level.

6.1.8 Refill Days vs. Anomaly Detection

Below shows how refill days are highlighted separately from true anomalies. This ensures that high-consumption refill events are not mistakenly flagged as critical, significant, or slight anomalies, improving the reliability of the system.

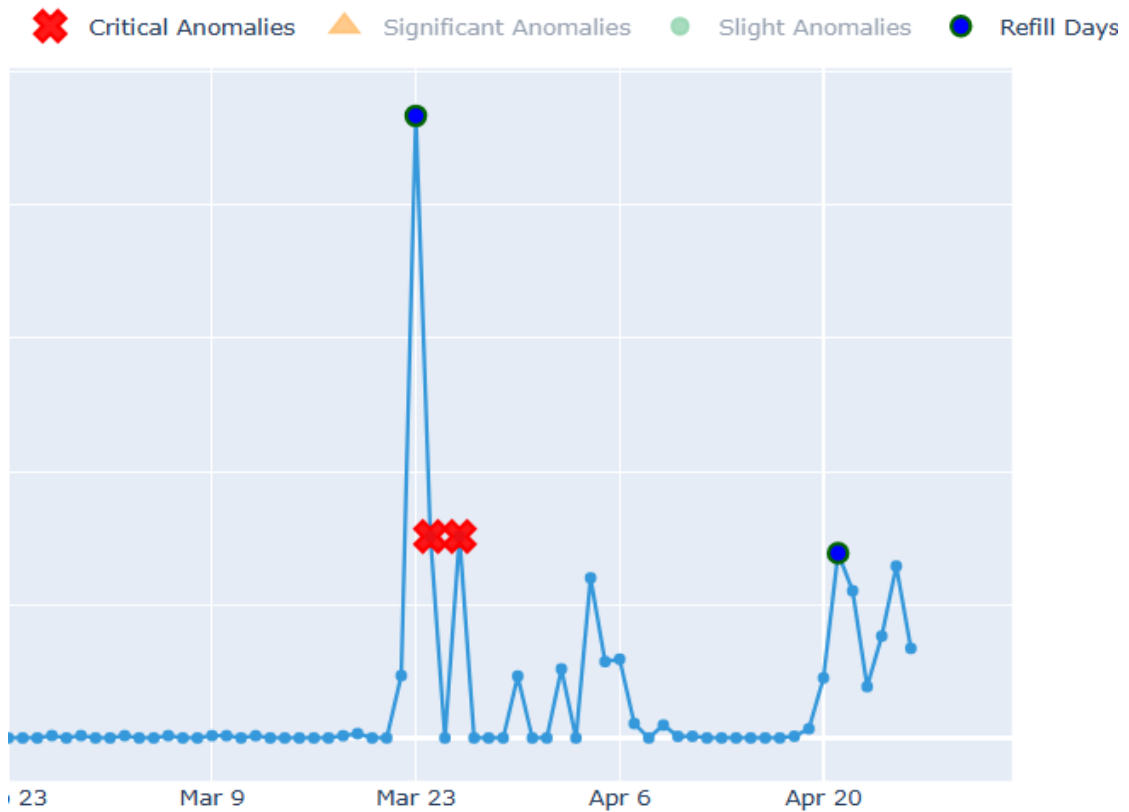


FIGURE 6.11: severity level.

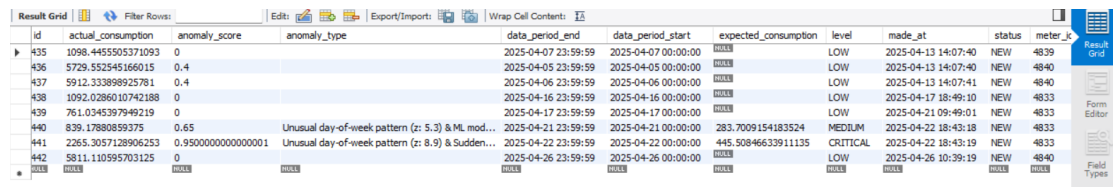
6.1.9 Saved Models

Previously saved models are successfully loaded, enabling the system to quickly move into predicting anomalies without retraining

```
Processing meter 4840 (2/14)...  
Using threshold 4.8000000000000025 for meter 4840  
WARNING:absl:Compiled the loaded model, but the compiled  
in or evaluate the model.  
Successfully loaded model for meter 4840  
Successfully loaded Isolation Forest for meter 4840  
Successfully loaded existing models for meter 4840  
Predicting anomalies...
```

FIGURE 6.12: previously saved models

Anomalies are automatically logged in a MySQL database, capturing details such as actual consumption, severity level, timestamps, and the associated meter ID.



id	actual_consumption	anomaly_score	anomaly_type	data_period_end	data_period_start	expected_consumption	level	made_at	status	meter_id
435	1098.445505371093	0		2025-04-07 23:59:59	2025-04-07 00:00:00	1098	LOW	2025-04-13 14:07:40	NEW	4839
436	5729.552545166015	0.4		2025-04-05 23:59:59	2025-04-05 00:00:00	1098	LOW	2025-04-13 14:07:40	NEW	4840
437	5912.333898925781	0.4		2025-04-06 23:59:59	2025-04-06 00:00:00	1098	LOW	2025-04-13 14:07:41	NEW	4840
438	1092.0286010742188	0		2025-04-16 23:59:59	2025-04-16 00:00:00	1098	LOW	2025-04-17 18:49:10	NEW	4833
439	761.0345397949219	0		2025-04-17 23:59:59	2025-04-17 00:00:00	1098	LOW	2025-04-21 09:49:01	NEW	4833
440	839.17880859375	0.65	Unusual day-of-week pattern (z: 5.3) & Ml mod...	2025-04-21 23:59:59	2025-04-21 00:00:00	283.7009154183524	MEDIUM	2025-04-22 18:43:18	NEW	4833
441	2265.3057128906253	0.9500000000000001	Unusual day-of-week pattern (z: 8.9) & Sudden...	2025-04-22 23:59:59	2025-04-22 00:00:00	445.50846633911135	CRITICAL	2025-04-22 18:43:19	NEW	4833
442	5811.110595703125	0		2025-04-26 23:59:59	2025-04-26 00:00:00	1098	LOW	2025-04-26 10:39:19	NEW	4840

FIGURE 6.13: Logged anomalies in the `smaart_meter_alarm` MySQL table showing anomaly scores, timestamps, severity and meter IDs.

This visual testing process enabled both real-time and retrospective analysis of detection performance.

6.2 Results

The final anomaly detection system was deployed across 14 smart meters, covering a wide range of building types including offices, libraries, community halls, and fire stations. While each building was evaluated independently due to variations in available data, the system flagged multiple anomalies, the majority of which were categorised as low severity. However, a subset were marked as significant or critical, often tied to weekend activity, refill cycle mismatches, or extreme deviations from expected patterns.

The results showed that combining LSTM Autoencoders, Isolation Forest, and rule-based logic helped achieve a broader and more reliable detection capability. The LSTM component was particularly effective at identifying subtle anomalies in stable environments like libraries. Isolation Forest offered quick unsupervised detection but occasionally flagged noisy outliers. The rule-based engine proved highly interpretable and accurate in buildings with clear operational patterns, such as offices closed on weekends, which can be seen in Figure 6.6.

Consensus logic between methods ensured that only the most confident detections were persisted, reducing false positives. High-confidence anomalies often showed agreement between LSTM and Isolation Forest, and were supported by statistical deviations or domain-specific rules.

The anomalies detected were reviewed through the visual dashboard, as presented in Figure 4.4, which highlighted consumption spikes, days of concern, and anomaly explanations. Logged anomalies were also written to the MySQL database with details including anomaly score, type, and severity. Figure 6.13, The comparison of detection methods is summarised in Figure 6.8. Notably, the highest detection confidence reached was 93%, which aligns with industry standards where reliable anomaly detection in industrial applications typically falls between 90% and 95%.

Method	Strengths	Limitations
LSTM Autoencoder	Detects subtle usage patterns and adapts over time	Requires large training data and is slower to process
Isolation Forest	Fast, unsupervised, and effective at catching strong outliers	Can flag normal variation as anomalies in noisy environments
Rule-based Engine	Easy to understand and works well with predictable behaviour (e.g. weekends)	Doesn't adapt to unexpected usage patterns or unseen behaviours

TABLE 6.1: Summary of Anomaly Detection Methods Used

Chapter 7

Discussion and Conclusions

7.1 Project Overview and Review

This project focuses on detecting unusual energy patterns across Cork County Council's 14 smart meters, bringing together machine learning, database management, and domain-specific logic. It supports better energy monitoring, reducing waste and costs.

The system processes raw meter readings directly to detect patterns and irregularities. Isolation Forest and Long Short Term Memory (LSTM) autoencoders identify anomalies, while rule-based checks cover sharp refills and missing periods. A hybrid scoring approach flags issues and generates plain-language explanations.

Python scripts handle data processing, model application, and anomaly explanation. MySQL manages the data, and weather data is pulled from an external API, with historical records backfilled where needed. A Flask server exposes APIs, while an interactive dashboard built with Dash and Plotly allows users to explore trends, view anomalies, and understand causes.

Building the dashboard required modular callback design to keep graphs responsive to user selections, combining consumption, anomalies, and weather data seamlessly. This strengthened skills in Python Dash development and visualisation structuring.

Training and tuning the machine learning models, especially using small, irregular datasets, required additional research during the semester break. Feature engineering, threshold setting, and hybrid scoring techniques were key to making the anomaly detection reliable and interpretable.

Overall, the project delivered a full working system, combining machine learning, data processing, database handling, and interactive visualisation in a way that supports real-world energy management.

7.2 Skills Gained

7.2.1 Python, Dash, Plotly

Building the dashboard using Python, Dash, and Plotly required designing a complete interactive system that could support real-time querying, filtering, and visualisation of smart meter data. Rather than using a simple Dash-only setup, I instead integrated Flask [15] as a server foundation, exposing dedicated API routes for different types of data such as meter information, daily readings, and anomalies. Within Dash [16], multiple callback functions were created to dynamically update graphs based on user-selected buildings, dates, and display options, allowing consumption trends, weather conditions, and anomaly patterns to be explored together on demand. Each visual layer, including seasonal trends, refills, weekends, and holidays, was carefully organised across multiple axes and subplots using Plotly [17] to make the analysis clear and intuitive. Building the system in a modular, class-based structure made it easier to manage complexity and ensured the application remained flexible and maintainable as more features were added.

7.2.2 Machine Learning Models

A major learning point was developing the anomaly detection system itself. The solution required building a layered approach that combined machine learning models with domain-specific logic. Data was first prepared by engineering new features from the raw meter readings, such as rolling averages, weekend and holiday flags, and seasonal trends. A Long Short Term Memory (LSTM) autoencoder [18] was then trained to capture the normal patterns of energy usage, while an Isolation Forest model [19] was used to detect more unexpected deviations. Alongside these, rule-based detectors were designed to catch specific issues like sharp refills or zero consumption periods. These different signals were then merged into a hybrid decision system, scoring the likelihood of an anomaly and automatically explaining the suspected causes in plain language. Developing this system involved even more significant research during the semester break to understand and learn how to build, train, and tune these models effectively. Training the models, setting detection thresholds, interpreting outputs, and validating results

across real-world smart meter data offered practical experience far beyond basic machine learning theory.

7.2.3 Project Management

Project management became one of the most important skills developed during the course of this work. Managing the system build across five sprints taught that even strong early plans must constantly adapt to the reality of technical problems and external pressures. Early in the project, progress moved smoothly, but setbacks soon appeared from both internal challenges, such as gaps in the weather data and the need to redesign the anomaly detection logic to lower false positives, and external factors, including a heavy Semester 2 coursework load, the effects of fasting during Ramadan, and a period of illness that reduced momentum.

Breaking the project into sprints with clear, realistic targets allowed better control over the work. Weekly goals were adjusted in response to new problems, whether retraining models when early anomaly detection outputs proved unreliable, or restructuring the dashboard callbacks when responsiveness issues emerged. Regular note-taking on obstacles and updating the Scrumban board to reflect true progress became vital to avoid falling behind.

The experience showed that no matter how well-designed a system is on paper, successful delivery depends on constant iteration, honest reassessment, and leaving enough space for rework and testing. Learning to balance technical depth with flexible planning under real-world pressures is one of the strongest skills carried forward from this project.

7.2.4 Area of Improvement

Several areas for improvement became clear through the project. One was not fully utilising GitHub's workflow features, such as structured issue tracking, version branching, and pull request reviews. I relied mainly on local development. A more disciplined use of GitHub from the beginning would have improved collaboration, documentation, and rollback options during testing.

Another improvement would have been integrating historical weather data earlier. Initially, only live weather readings were collected, but later experience showed that having historical weather trends from the start would have strengthened the feature set and improved model explanations. However, attempts to find reliable free sources for historical weather proved unsuccessful, as most services offering full past datasets were paid, limiting the ability to integrate real historical weather into the early phases of the project.

Establishing a formal testing framework early in the anomaly detection code would also have helped. While functional testing was done manually, setting up automated validation from the start would have caught subtle errors sooner and enabled faster iteration across different model tuning experiments.

Better planning around academic workload fluctuations and personal commitments, such as Semester 2 coursework deadlines, fasting during Ramadan, and a period of illness, would have avoided late project compression. Building larger time buffers into the sprint planning and allowing more slack for unexpected external setbacks would have led to a more balanced, less pressured final phase.

Finally, if I were to carry out a similar project again, I would place much more focus on doing better research earlier in Semester 1 to make sure the idea is fully transparent and easy for external examiners to follow. Looking back, I don't think I was very clear during the research phase — writing reports has never been one of my strengths, but I feel like I improved a lot at it during Semester 2. I would also aim to structure the code in a more modular way from the start, integrate supporting datasets earlier, and set up a proper testing framework from the beginning. Testing the models in smaller, more frequent cycles would have helped identify issues sooner and made the whole system easier to refine.

7.2.5 User Feedback

Following the deployment of the smart meter anomaly detection system for Cork County Council, we conducted user interviews to evaluate the practical utility and effectiveness of the system. The feedback received provides valuable insights into how the system is being used in real-world conditions and identifies areas for future enhancement.

7.2.5.1 Positive Feedback

Users consistently highlighted the following strengths of the system:

- **Fulfillment of Smart Meter Potential:** Multiple users indicated that the anomaly detection system addresses the fundamental purpose of smart meter installations, with one user noting, “This is exactly why smart meters are out there—to actually use the data they collect in meaningful ways.”
- **Weekend and Nighttime Detection:** The system’s ability to identify unusual consumption patterns during weekends and night hours was specifically praised. Building managers found this particularly valuable as these time periods typically have minimal occupancy and thus any significant consumption likely indicates issues requiring attention.
- **Unexpected Pattern Recognition:** Users appreciated the system’s capability to detect anomalies that would not have been anticipated by human observers. As one user explained, “The system catches patterns we wouldn’t have thought to look for, which has helped us identify several inefficiencies in our building systems.”
- **Alert Accuracy:** Feedback confirmed that the anomaly alerts generated by the system demonstrated high accuracy. False positives were minimal, increasing user confidence in the system’s recommendations.
- **Value of Expected Consumption Estimates:** The inclusion of expected consumption values alongside detected anomalies was highlighted as particularly useful for contextualizing the significance of deviations and prioritizing response actions.

7.3 Conclusion

This project set out to address the challenge of detecting unusual energy consumption patterns across Cork County Council's 14 smart meters. The background research confirmed that smart meter data is often noisy, seasonal, and strongly affected by external factors like weather and building usage schedules. Existing anomaly detection approaches such as Isolation Forests and LSTM autoencoders were studied, but it became clear that a hybrid system combining machine learning models with domain-specific rules would offer the best solution.

The final system brings together several layers: a MySQL database for secure and efficient data storage, Python-based scripts for feature engineering and anomaly scoring, and a Flask and Dash web application to allow real-time exploration of energy trends and detected issues. Consumption readings are processed to create meaningful features, such as rolling averages, seasonality effects, and event flags for weekends, holidays, and refills. Anomaly detection then uses a combination of an LSTM autoencoder, Isolation Forest, and rule-based triggers to identify different types of irregularities.

Primary conclusions drawn from the project are that no single model is sufficient for smart meter anomaly detection in isolation; combining multiple techniques increases accuracy and reduces false positives. Another important conclusion is that integrating external information, such as weather data, even if partially synthetic, helps explain anomalies in a more understandable way.

A secondary conclusion is the importance of system modularity. Structuring the solution with clean separation between data ingestion, processing, machine learning, and frontend interaction allowed each part to be tested and improved independently. This flexibility proved critical when late adjustments were needed, such as refining detection thresholds or redesigning graph interactions.

Overall, the project delivers a working, scalable, and understandable system that can assist Cork County Council in managing energy usage more intelligently.

7.4 Future Work

Although the project meets its main objectives, there are several areas where improvements and enhancements could have been made if more time were available.

- **Anomaly Notification System:** Given more time, I would have extended the system to automatically notify users when anomalies are detected, rather than relying on manual dashboard monitoring. This would allow faster reactions to irregular energy patterns and make the system more proactive.
- **Integration of Real Historical Weather Data:** I would have sourced real historical weather data instead of relying on synthetic backfills. However, during the project, it became clear that free reliable sources were limited, and most real datasets required paid access. With more time and resources, integrating authentic weather records would improve the quality of anomaly explanations.
- **Expansion of Anomaly Types:** Another improvement would involve expanding the types of anomalies detected, such as spotting long-term drifts in baseline consumption, detecting abnormal weekend-weekday transitions, or capturing patterns specific to certain building operations.
- **Migration to a Scalable Frontend:** If additional time were available, I would have considered moving away from Dash and Plotly towards a more scalable web framework, better suited for high user loads and real-time responsiveness.
- **Periodic Model Retraining:** If the project were extended, I would propose implementing a retraining pipeline for the anomaly detection models. As more smart meter data becomes available and new usage patterns emerge, retraining the LSTM and Isolation Forest models every six months would help maintain detection accuracy and adapt the system to ongoing changes in energy consumption behaviour.
- **Finer Temporal Granularity:** Based on user feedback, implementing anomaly detection at a finer level than daily would be valuable. Users specifically requested anomaly detection capabilities at an hourly or sub-hourly level to better pinpoint when during the day abnormal consumption occurs.

Bibliography

- [1] Businessmap.io, “Scrumban methodology,” 2023, accessed: 2024-10-30. [Online]. Available: <https://businessmap.io/blog/scrumban>
- [2] Diagrams.net, “Diagrams.net online diagram tool,” 2023, accessed: 2024-10-30. [Online]. Available: <https://app.diagrams.net/>
- [3] Y. Himeur, K. Ghanem, A. Alsalemi, F. Bensaali, and A. Amira, “Artificial intelligence-based anomaly detection of energy consumption in buildings: A review, current trends, and new perspectives,” *Applied Energy*, vol. 287, p. 116601, 2021. [Online]. Available: https://www.researchgate.net/publication/349193113_Artificial_intelligence_based_anomaly_detection_of_energy_consumption_in_buildings_A_review_current_trends_and_new_perspectives
- [4] J. Lee, J. Kim, and H. Park, “Anomaly detection of smart metering system for power management with battery storage system,” *ETRI Journal*, vol. 44, no. 5, pp. 678–687, 2022.
- [5] H. Chahla, Y. Lafifi, and M. Bouzidi, “Hybrid model for anomaly detection in power consumption data,” *IEEE Transactions on Smart Grid*, vol. 10, no. 3, pp. 2302–2311, 2019.
- [6] A. Aligholian, M. Farajollahi, and H. Mohsenian-Rad, “Unsupervised learning for online abnormality detection in smart meter data,” *IEEE Transactions on Smart Grid*, vol. 10, no. 6, pp. 6789–6798, 2019.
- [7] T. Vafeiadis, A. Alexiadis, V. Dimaridou, S. Krinidis, K. Kitsikoudis, L. Makris, D. Davidović, D. Ioannidis, and D. Tzovaras, “Anomaly detection in smart meters using autoencoders,” in *14th Conference of Slovenian Electrical Power Engineers (CIRED)*, 2019.
- [8] M. Khoshsefat, A. Rahdar, H. A. Ravaghi Ardabili, H. Karimi, A. Sheykhaleh, and D. Gharavian, “Anomaly detection in electricity consumption data: A cnn-based approach for district-level analysis in tehran, iran,” in *28th Electrical Power Distribution Conference (EPDC)*, 2024.

- [9] D. Singhal, L. Ahuja, and A. Seth, “Anomaly detection in smart meters: Analytical study,” *Proceedings of the 2nd International Conference on Power Electronics and IoT Applications in Renewable Energy and Its Control (PARC)*, pp. 1–4, 2022.
- [10] P. S. Foundation, “Python programming language,” [Online]. Available: <https://www.python.org/>, 2023.
- [11] O. Corporation, “Mysql database,” [Online]. Available: <https://www.mysql.com/>, 2023.
- [12] GitHub, “Github docs,” [Online]. Available: <https://docs.github.com/en/get-started/quickstart/hello-world>, 2023.
- [13] R. Contributors, “React documentation,” [Online]. Available: <https://reactjs.org/>, 2023.
- [14] M. D. Network, “Html and css documentation,” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>, 2023.
- [15] A. Ronacher, “Flask documentation,” 2023, accessed: 2024-11-24. [Online]. Available: <https://flask.palletsprojects.com/en/2.3.x/>
- [16] P. T. Inc., “Dash user guide and documentation,” 2023, accessed: 2024-11-24. [Online]. Available: <https://dash.plotly.com/introduction>
- [17] —, “Plotly python graphing library,” 2023, accessed: 2024-11-24. [Online]. Available: <https://plotly.com/python/>
- [18] K. Documentation, “Lstm layers,” 2023, accessed: 2024-11-24. [Online]. Available: https://keras.io/api/layers/recurrent_layers/lstm/
- [19] S. learn Developers, “Isolation forest — anomaly detection,” 2023, accessed: 2024-11-24. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>

Appendix A

Code Snippets

```
def get_db_connection(self):  
    """  
    ...  
    Create and return a database connection  
    ...  
    Returns:  
    ... pymysql.Connection: Database connection object  
    ...  
    """  
    conn = pymysql.connect(  
        ... host=self.host,  
        ... user=self.user,  
        ... password=self.password,  
        ... db=self.db,  
        ... port=self.port  
    )  
    ... return conn
```

FIGURE A.1: Function to establish and return a MySQL database connection using the stored configuration

```

def log_anomaly_to_database(self, anomaly_row):
    # Define the cutoff date as a Timestamp object.
    cutoff_date = pd.Timestamp('2025-04-01')
    # If the anomaly date is before the cutoff, do not log it.
    if anomaly_row['date'] < cutoff_date:
        print(f"Skipping logging anomaly for meter {anomaly_row['meter_id']} on {anomaly_row['date']}, "
              f"as it is before {cutoff_date.date()}")
        return False

    try:
        # Get connection
        conn = self.get_db_connection()
        cursor = conn.cursor()

        # Convert and extract required fields
        meter_id = str(anomaly_row['meter_id'])

        # Handle the consumption value correctly -- remove 'kwh' if present and convert to float
        consumption_str = str(anomaly_row['daily_consumption'])
        if 'kwh' in consumption_str:
            consumption_str = consumption_str.replace('kwh', '').strip()
        actual_consumption = float(consumption_str)

        anomaly_score = float(anomaly_row.get('anomaly_score', 0.0))
        anomaly_type = str(anomaly_row.get('anomaly_type', 'Unknown'))[:255]

```

FIGURE A.2: Function to log anomaly data to the database after validating the anomaly date and parsing fields

```

def create_lstm_autoencoder(self, input_shape):
    """Enhanced LSTM architecture with BatchNorm and Dropout"""
    inputs = tf.keras.layers.Input(shape=(input_shape[1], input_shape[2]))

    # Encoder
    x = tf.keras.layers.BatchNormalization()(inputs)
    x = tf.keras.layers.LSTM(64, activation='relu', return_sequences=True)(x)
    x = tf.keras.layers.Dropout(0.2)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.LSTM(32, activation='relu', return_sequences=True)(x)
    x = tf.keras.layers.Dropout(0.2)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    encoded = tf.keras.layers.LSTM(16, activation='relu')(x)

    # Decoder
    x = tf.keras.layers.RepeatVector(input_shape[1])(encoded)
    x = tf.keras.layers.LSTM(16, activation='relu', return_sequences=True)(x)
    x = tf.keras.layers.Dropout(0.2)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.LSTM(32, activation='relu', return_sequences=True)(x)
    x = tf.keras.layers.Dropout(0.2)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.LSTM(64, activation='relu', return_sequences=True)(x)
    decoded = tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(input_shape[2]))(x)

```

FIGURE A.3: LSTM autoencoder model with Batch Normalization and Dropout for enhanced anomaly detection

```
# Global mapping for meter IDs to building names
METER_TO_BUILDING = {
    '4663': 'Castletownbere Area Engineers Office',
    '4664': 'Newberry Cross Machinery Depot',
    '4665': 'Skibbereen MD',
    '4832': 'Clonakilty Library & Offices',
    '4833': 'Skibbereen Library',
    '4834': 'Skibbereen Heritage Centre',
    '4835': 'Environmental Office, Inniscarra',
    '4836': 'Mitcheltown Area Engineers Office',
    '4837': 'Kinsale MD Office',
    '4839': 'Bantry Fire Station',
    '4840': 'Charleville Area Engineers Office',
    '4841': 'Clonakilty MD Office - Town Hall',
    '4842': 'Castletownbere Area Engineers Office',
    '4859': 'Macroon MD, AEO and Community Hall'
}
```

FIGURE A.4: Global dictionary mapping meter IDs to corresponding building names for identification

```
# Process each day with improved thresholds
for idx, row in meter_data.iterrows():
    anomaly_score = 0.0
    anomaly_types = []
    confidence_factors = []
    detection_methods = []

    # Skip anomaly detection if this is a refill day
    if not row['is_refill']:
        # 1. Z-score based detection with building-specific threshold
        if 'zscore' in row and pd.notna(row['zscore']) and abs(row['zscore']) > threshold: # Use building-specific threshold
            anomaly_score += 0.4 # Reduced contribution from 0.5
            anomaly_types.append(f'Statistical outlier (z-score: {row["zscore"]:.1f})')
            confidence_factors.append(min(1.0, abs(row['zscore']) / 5))
            detection_methods.append('zscore')

        # 2. Day-of-week pattern with building-specific threshold
        if 'day_of_week' in row and pd.notna(row['day_of_week']):
            dow_data = meter_data[meter_data['day_of_week'] == row['day_of_week']]
            if len(dow_data) >= 4: # Need more examples for stability
                dow_mean = dow_data['daily_consumption'].mean()
                dow_std = dow_data['daily_consumption'].std()
                if dow_std > 0:
                    dow_zscore = abs(row['daily_consumption'] - dow_mean) / dow_std
                    if dow_zscore > threshold: # Use building-specific threshold
                        anomaly_score += 0.25 # Reduced from 0.3
                        anomaly_types.append(f'Unusual day-of-week pattern (z: {dow_zscore:.1f})')
                        confidence_factors.append(min(1.0, dow_zscore / 4.0))
                        detection_methods.append('day_pattern')
```

FIGURE A.5: Daily anomaly detection logic using z-score and day-of-week pattern with building-specific thresholds


```

def add_domain_knowledge_rules(self, df):
    print("Applying domain knowledge rules with building-specific parameters...")

    # Loop through each building type separately
    for meter_id, building_name in METER_TO_BUILDING.items():
        meter_data = df[df['meter_id'] == meter_id]
        if len(meter_data) < 14: # Skip if not enough data for reliable patterns
            continue

        # Determine building type and apply appropriate rules
        building_type = self._classify_building_type(building_name)

        # Office buildings
        if building_type == 'office':
            # Analyze typical office hours pattern (weekday 9-5)
            weekday_avg = meter_data[meter_data['is_weekend'] == 0]['daily_consumption'].mean()
            weekday_std = meter_data[meter_data['is_weekend'] == 0]['daily_consumption'].std()

            # Weekend detection -- more precise by understanding this specific building
            weekend_historic = meter_data[meter_data['is_weekend'] == 1]['daily_consumption']
            if len(weekend_historic) >= 4:
                weekend_avg = weekend_historic.mean()
                weekend_std = weekend_historic.std()
                weekend_threshold = max(
                    weekend_avg + 1.5 * weekend_std, # Statistical approach
                    weekday_avg * 0.7 # Percentage of weekday usage
                )

```

FIGURE A.6: Function to apply domain knowledge rules by classifying building types and analysing consumption patterns

```

def generate_anomaly_explanation(self, anomaly_row):
    explanation = []

    # Get building name if available
    building_name = METER_TO_BUILDING.get(str(anomaly_row['meter_id']), f"Building: {anomaly_row['meter_id']}")

    # Get day name
    day_names = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
    day_name = day_names[anomaly_row['day_of_week']]

    # Basic information with more direct formatting
    explanation.append(f"Building: {building_name}")
    explanation.append(f>Date: {anomaly_row['date'].strftime('%A, %B %d, %Y')}")
    explanation.append(f"Consumption: {anomaly_row['daily_consumption']:.2f} kWh")

    # Expected consumption with clearer difference metrics
    if not pd.isna(anomaly_row['expected_consumption']):
        # Calculate percent and absolute difference
        pct_diff = ((anomaly_row['daily_consumption'] - anomaly_row['expected_consumption']) /
                    max(0.01, anomaly_row['expected_consumption']) * 100)
        abs_diff = anomaly_row['daily_consumption'] - anomaly_row['expected_consumption']

        explanation.append(f"Expected: {anomaly_row['expected_consumption']:.2f} kWh")
        explanation.append(f"Difference: {abs_diff:.2f} kWh ({pct_diff:.1f}% {'higher' if pct_diff > 0 else 'lower'})")

```

FIGURE A.7: Function to generate a human-readable explanation of anomalies based on consumption deviation

Appendix B

Wireframe Models

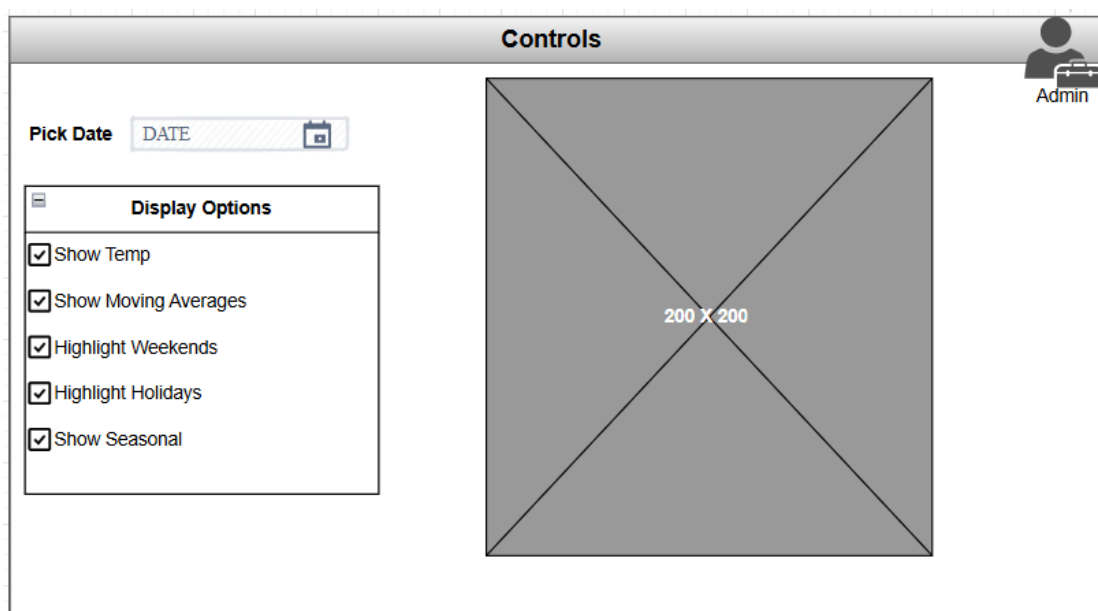


FIGURE B.1: Wireframe layout of the control panel interface

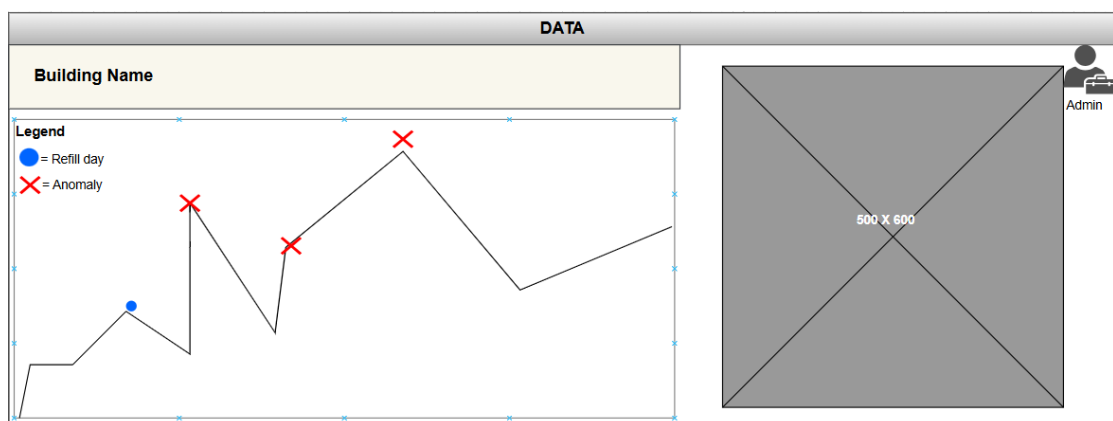


FIGURE B.2: Wireframe showing data panel with anomaly and refill indicators

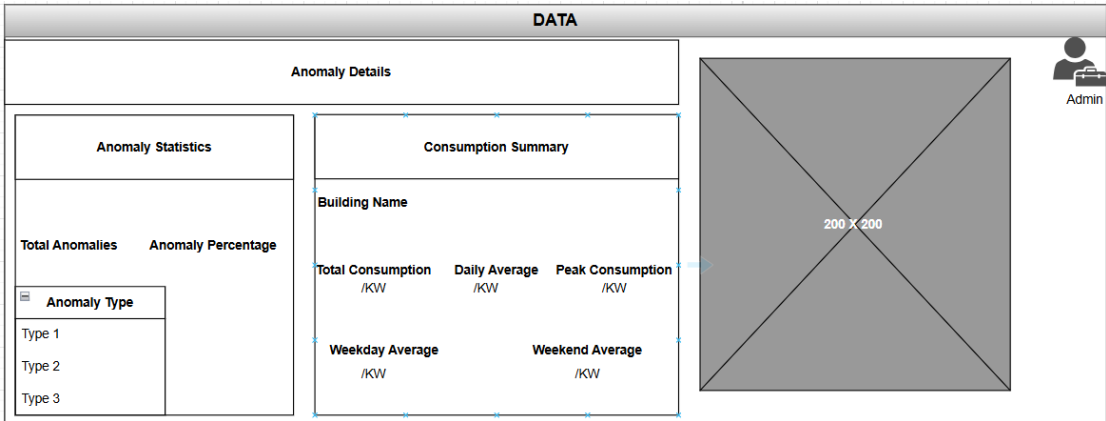


FIGURE B.3: Wireframe showing anomaly statistics and consumption summary panel

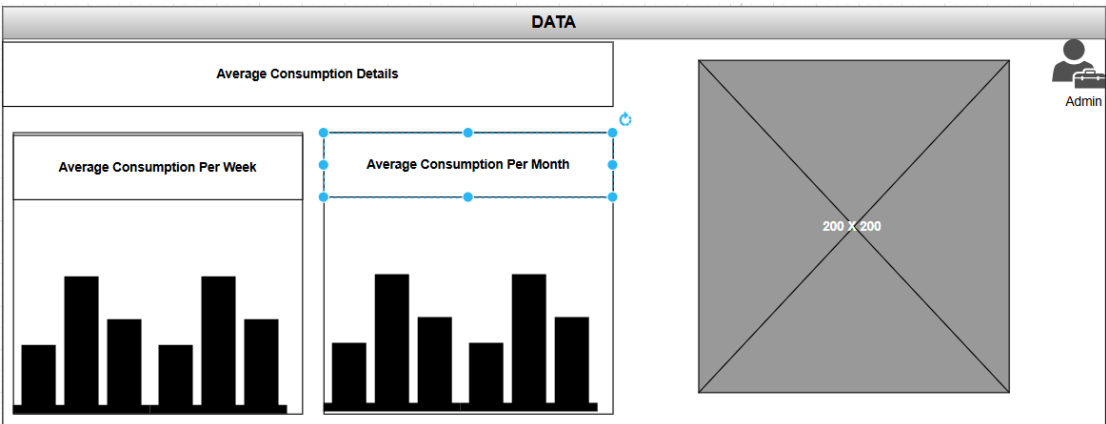


FIGURE B.4: Wireframe showing average weekly and monthly consumption details

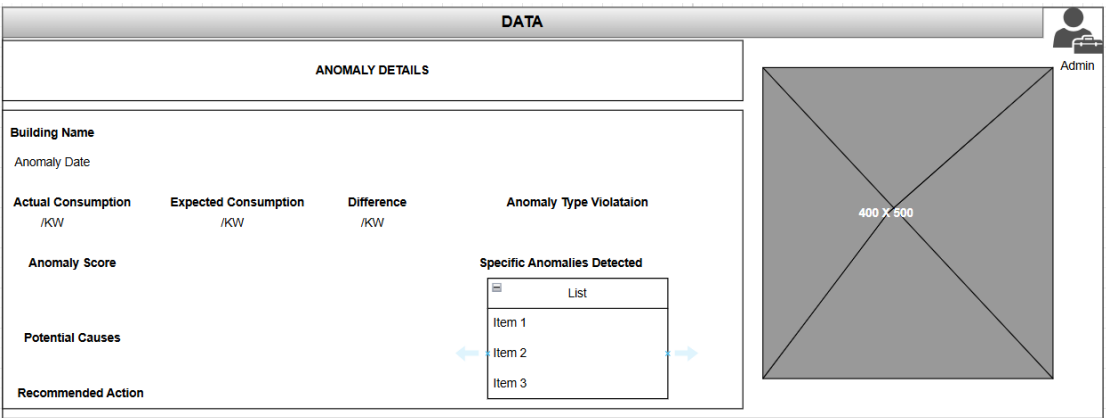


FIGURE B.5: Wireframe showing anomaly details including consumption metrics and detected issues